📦 Cell 1: Load Dataset

Purpose: Load the student depression dataset using pandas.

Explanation:

- `pandas` is imported for data manipulation.
- The dataset is read using `read_csv()`.

Observation: The dataset is now loaded into a DataFrame named `df`.

Insight: This is the foundational step before doing any analysis.

```
import pandas as pd

# Step 1: Load the CSV file
df = pd.read_csv("Student Depression Dataset.csv")  # Replace with your actual filename

# Step 2: View first few rows
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Deg
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Ph
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	

👀 Cell 2: View First 5 Rows

Purpose: Preview the top of the dataset.

Explanation: `df.head()` shows the first 5 rows, helping us understand the data structure.

Observation: Allows us to check column names, data types, and possible missing or inconsistent data early on.

```
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Deg
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Ph
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	

🔚 Cell 3: View Last 5 Rows

Purpose: Check the last few entries in the dataset.

Explanation: `df.tail()` displays the last 5 rows. Useful to ensure data consistency at the end of the dataset.

Insight: Helps verify completeness and detect any end-of-file issues.

```
df.tail()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits
27896	140685	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhealthy
27897	140686	Male	27.0	Ludhiana	Student	2.0	0.0	9.40	3.0	0.0	Less than 5 hours	Healthy
27898	140689	Male	31.0	Faridabad	Student	3.0	0.0	6.61	4.0	0.0	5-6 hours	Unhealthy
27899	140690	Female	18.0	Ludhiana	Student	5.0	0.0	6.88	2.0	0.0	Less than 5 hours	Healthy

📏 Cell 4: Dataset Shape

Purpose: Find out how many rows and columns the dataset has.

Explanation: `df.shape` returns a tuple with the number of rows and columns.

Insight: This gives a quick understanding of dataset size — important for memory and analysis planning.

```
# Step 3: Check the shape (rows, columns)
print("Shape:", df.shape)
```

```
Shape: (27901, 18)
```

🧠 Cell 5: Data Types and Null Values

Purpose: Get metadata about the dataset: column names, non-null counts, and data types.

Explanation: `df.info()` is crucial for identifying data issues such as missing values and incorrect data types.

Insight: Guides how to clean or transform data in the next steps.

```
# Step 4: Get info about data types and non-null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27901 entries, 0 to 27900
Data columns (total 18 columns):
 #   Column                                 Non-Null Count  Dtype  
---  ------                                 --------------  -----  
 0   id                                     27901 non-null  int64  
 1   Gender                                 27901 non-null  object 
 2   Age                                    27901 non-null  float64
 3   City                                   27901 non-null  object 
 4   Profession                             27901 non-null  object 
 5   Academic Pressure                      27901 non-null  float64
 6   Work Pressure                          27901 non-null  float64
 7   CGPA                                   27901 non-null  float64
 8   Study Satisfaction                     27901 non-null  float64
 9   Job Satisfaction                       27901 non-null  float64
 10  Sleep Duration                         27901 non-null  object 
 11  Dietary Habits                         27901 non-null  object 
 12  Degree                                 27901 non-null  object 
 13  Have you ever had suicidal thoughts ?  27901 non-null  object 
 14  Work/Study Hours                       27901 non-null  float64
 15  Financial Stress                       27898 non-null  float64
 16  Family History of Mental Illness      27901 non-null  object 
 17  Depression                             27901 non-null  int64  
dtypes: float64(8), int64(2), object(8)
memory usage: 3.8+ MB
```

```
# Step 5: Get basic statistics (mean, std, min, max, etc.)
df.describe()
```

	id	Age	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Work/Study Hours	Fina S
count	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27898.0
mean	70442.149421	25.822300	3.141214	0.000430	7.656104	2.943837	0.000681	7.156984	3.1
std	40641.175216	4.905687	1.381465	0.043992	1.470707	1.361148	0.044394	3.707642	1.4
min	2.000000	18.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.0
25%	35039.000000	21.000000	2.000000	0.000000	6.290000	2.000000	0.000000	4.000000	2.0
50%	70684.000000	25.000000	3.000000	0.000000	7.770000	3.000000	0.000000	8.000000	3.0
75%	105818.000000	30.000000	4.000000	0.000000	8.920000	4.000000	0.000000	10.000000	4.0

```
df.isnull().sum()
```

```
id                                     0
Gender                                 0
Age                                    0
City                                   0
Profession                             0
Academic Pressure                      0
Work Pressure                          0
CGPA                                   0
Study Satisfaction                     0
Job Satisfaction                       0
Sleep Duration                         0
Dietary Habits                         0
Degree                                 0
Have you ever had suicidal thoughts ?  0
Work/Study Hours                       0
Financial Stress                       3
Family History of Mental Illness       0
Depression                             0
dtype: int64
```

```
df['Depression'].value_counts()
```

```
Depression
1    16336
0    11565
Name: count, dtype: int64
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

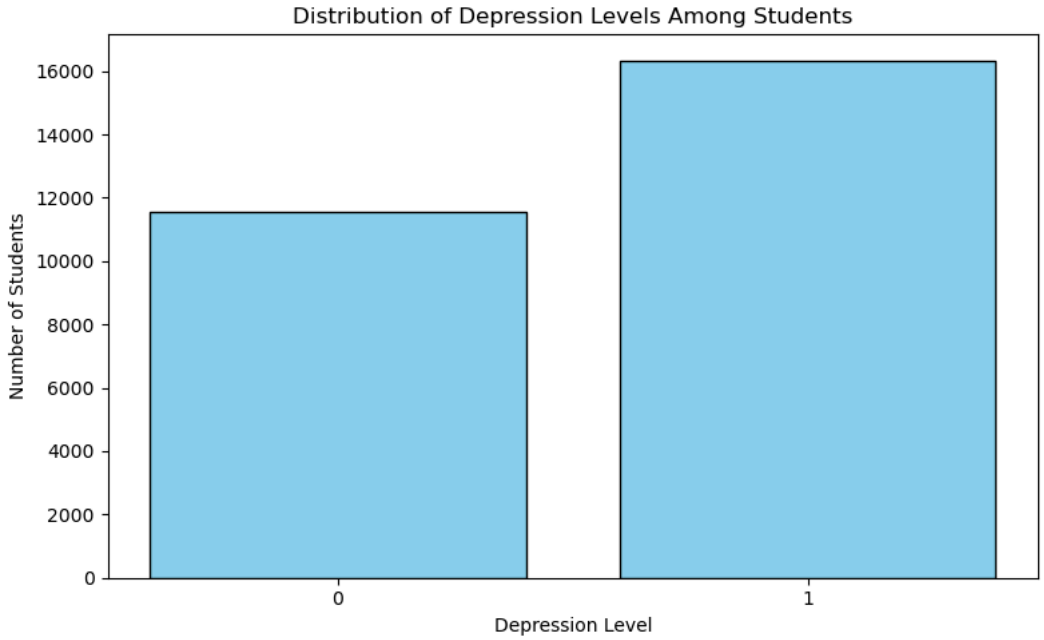
# Clean the Depression column (remove any extra spaces or unexpected formatting)
df['Depression'] = df['Depression'].astype(str).str.strip()

# Get value counts sorted by level (x-axis)
depression_counts = df['Depression'].value_counts().sort_index()

# Plot the bar chart
plt.figure(figsize=(8, 5))
plt.bar(depression_counts.index, depression_counts.values, color='skyblue', edgecolor='black')

# Add labels and title
plt.title('Distribution of Depression Levels Among Students')
plt.xlabel('Depression Level')
plt.ylabel('Number of Students')
plt.xticks(rotation=0)

# Display the graph
plt.tight_layout()
plt.show()
```




```
# checking the distribution of Target Variable according to gender 0 = female and 1 = male
df.groupby('Gender')['Depression'].value_counts()
```

```
Gender  Depression
Female  1             7221
        0             5133
Male    1             9115
        0             6432
Name: count, dtype: int64
```

```
#group the dataframe on target to estimate the mean of every column.
import pandas as pd

df = pd.read_csv("Student Depression Dataset.csv")

# Group by 'Depression' and get mean of numeric columns
result = df.groupby('Depression').mean(numeric_only=True)

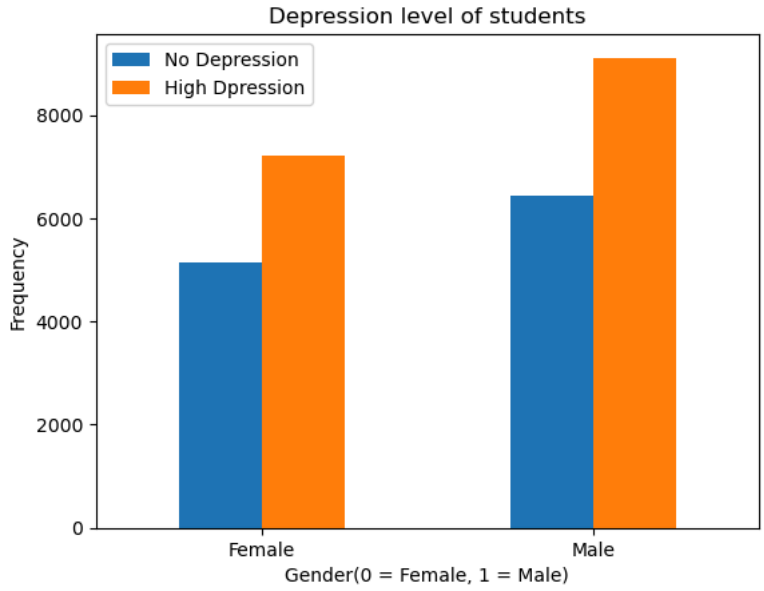
print(result)
```

```
                      id        Age  Academic Pressure  Work Pressure  \
Depression
0           70397.561089  27.142412           2.361608       0.000605
1           70473.715536  24.887733           3.693132       0.000306

                CGPA  Study Satisfaction  Job Satisfaction  Work/Study Hours  \
Depression
0           7.617282            3.215564          0.000865          6.237959
1           7.683588            2.751469          0.000551          7.807603

            Financial Stress
Depression
0                   2.518724
1                   3.579553
```

```
#to visualize disease frequency according to gender
pd.crosstab(df.Gender,df.Depression).plot(kind="bar")
plt.title('Depression level of students')
plt.xlabel('Gender(0 = Female, 1 = Male)')
plt.xticks(rotation=0)
plt.legend(["No Depression ", "High Dpression"])
plt.ylabel('Frequency')
plt.show()
```



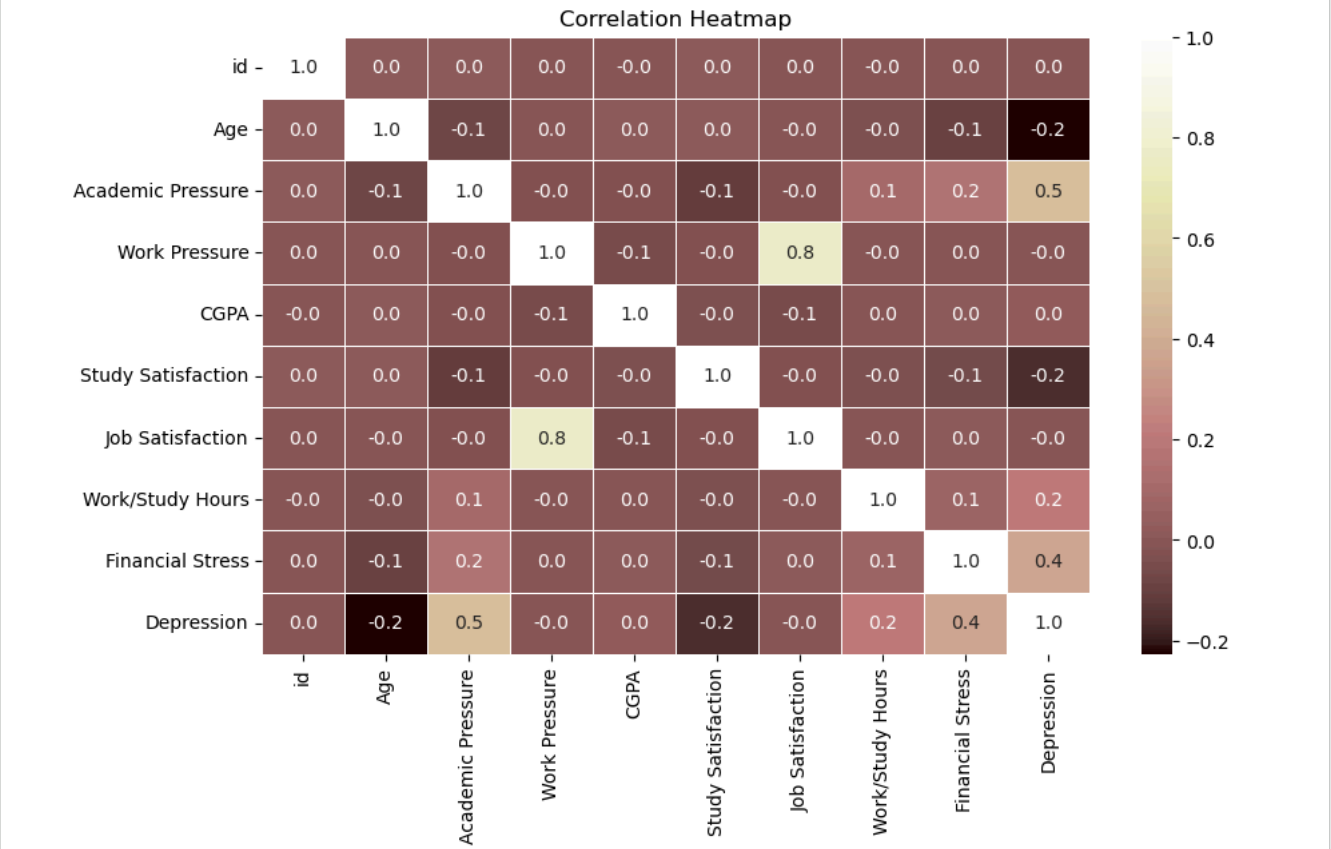

```
#to calculate Correlation of each feature
(df.select_dtypes(include='number').corr())
```

	id	Age	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Work/Study Hours	Financial Stress	Depression
id	1.000000	0.003783	0.005172	0.001261	-0.012324	0.007794	0.001931	-0.004466	0.000783	0.000923
Age	0.003783	1.000000	-0.075803	0.002015	0.005056	0.009235	-0.000432	-0.032928	-0.095026	-0.226422
Academic Pressure	0.005172	-0.075803	1.000000	-0.022231	-0.022239	-0.110988	-0.024945	0.095970	0.151706	0.474835
Work Pressure	0.001261	0.002015	-0.022231	1.000000	-0.050896	-0.021145	0.770652	-0.005468	0.001883	-0.003351
CGPA	-0.012324	0.005056	-0.022239	-0.050896	1.000000	-0.044062	-0.053631	0.002604	0.005885	0.022210
Study Satisfaction	0.007794	0.009235	-0.110988	-0.021145	-0.044062	1.000000	-0.021907	-0.036441	-0.065107	-0.167971
Job Satisfaction	0.001931	-0.000432	-0.024945	0.770652	-0.053631	-0.021907	1.000000	-0.005222	0.005248	-0.003482

```
#to visualize Correlation of each feature
import seaborn as sns
import matplotlib.pyplot as plt

# Only select numeric columns
corr = df.select_dtypes(include='number').corr()

# Plot heatmap
plt.figure(figsize=(10, 6))  # Adjust the size as needed
sns.heatmap(corr, annot=True, cmap='pink', fmt=".1f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```



Double-click (or enter) to edit

```
import pandas as pd
import matplotlib.pyplot as plt

# Set figure size before plotting
plt.figure(figsize=(24, 10))  # Increased width

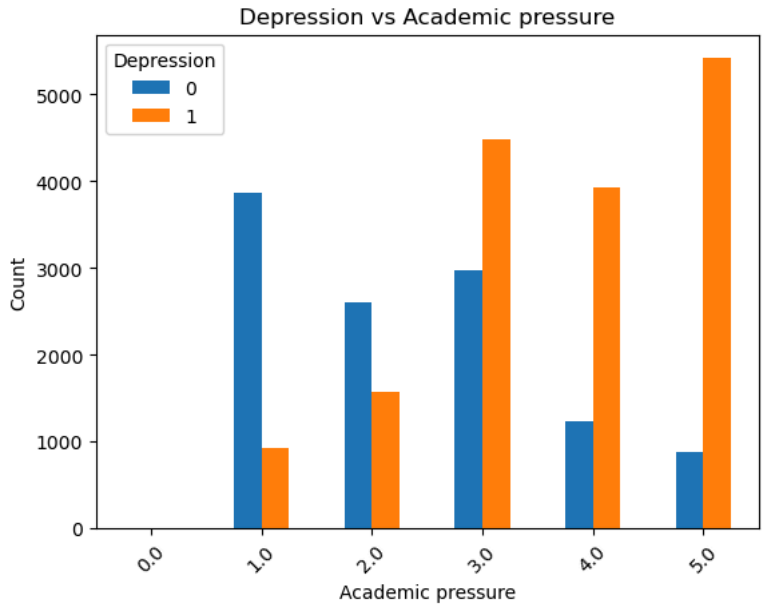
# Create the bar plot
pd.crosstab(df['Academic Pressure'], df['Depression']).plot(kind='bar')

# Add title and axis labels
plt.title('Depression vs Academic pressure')
plt.xlabel('Academic pressure')
plt.ylabel('Count')

# Rotate x-axis labels if needed
plt.xticks(rotation=45)

# Show the plot
plt.show()
```

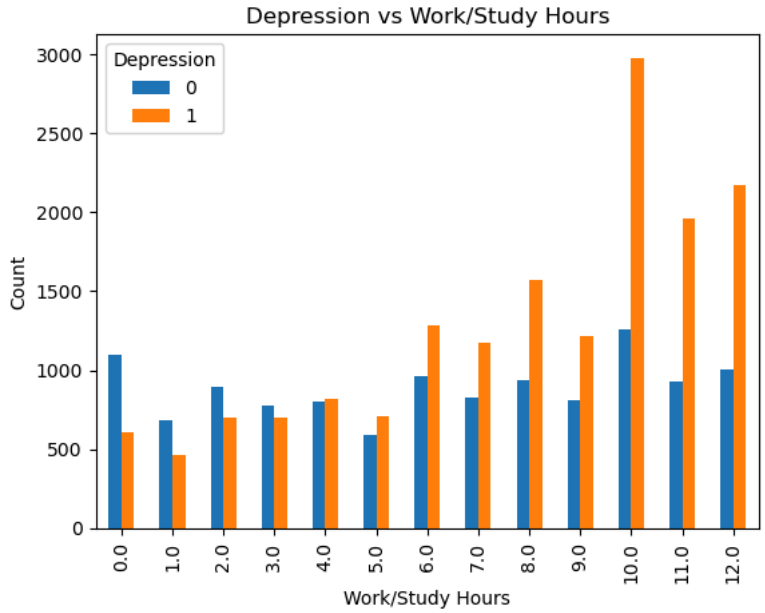
```
<Figure size 2400x1000 with 0 Axes>
```



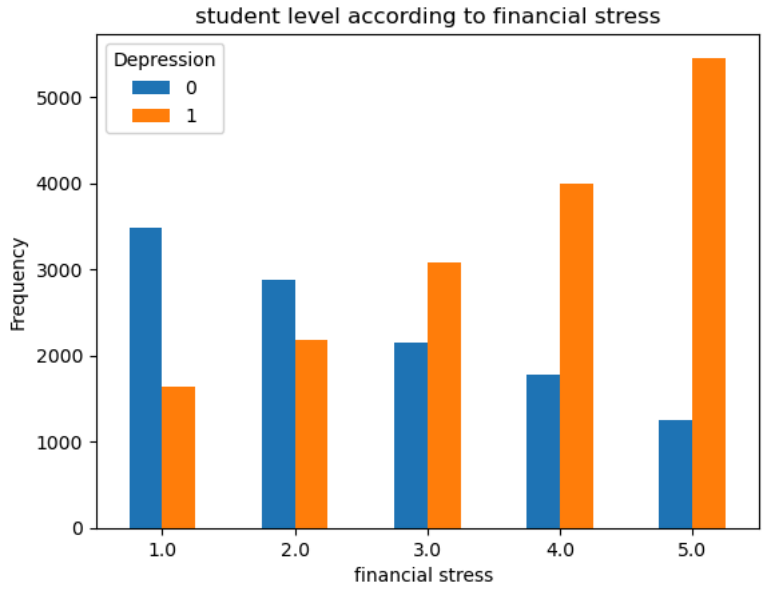

```
#to visualize disease frequency according to work pressure
import pandas as pd
import matplotlib.pyplot as plt

# Simple bar plot: Depression Level vs Work/Study Hours
pd.crosstab(df['Work/Study Hours'], df['Depression']).plot(kind='bar')

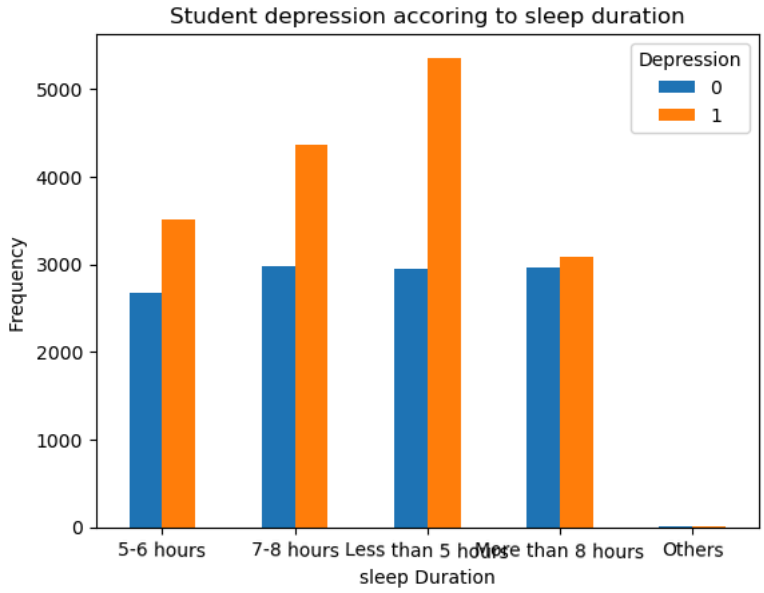
plt.title('Depression vs Work/Study Hours')
plt.xlabel('Work/Study Hours')
plt.ylabel('Count')
plt.show()
```




```
#to visualize disease frequency according to The Slope
pd.crosstab(df['Financial Stress'],df.Depression).plot(kind="bar")
plt.title('student level according to financial stress')
plt.xlabel('financial stress ')
plt.xticks(rotation = 0)
plt.ylabel('Frequency')
plt.show()
```

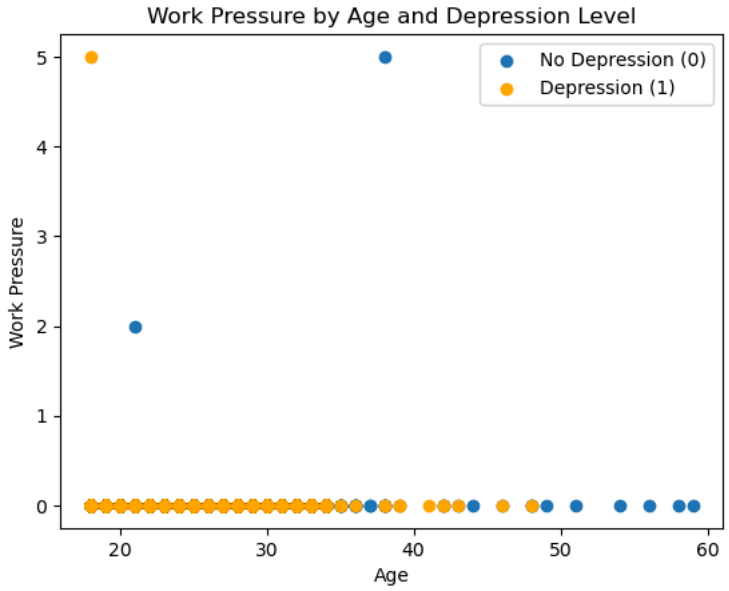



```
#to visualize disease frequency according to Thalassemia Type
pd.crosstab(df['Sleep Duration'],df.Depression).plot(kind="bar")
plt.title('Student depression accoring to sleep duration')
plt.xlabel('sleep Duration')
plt.xticks(rotation = -1)
plt.ylabel('Frequency ')
plt.show()
```

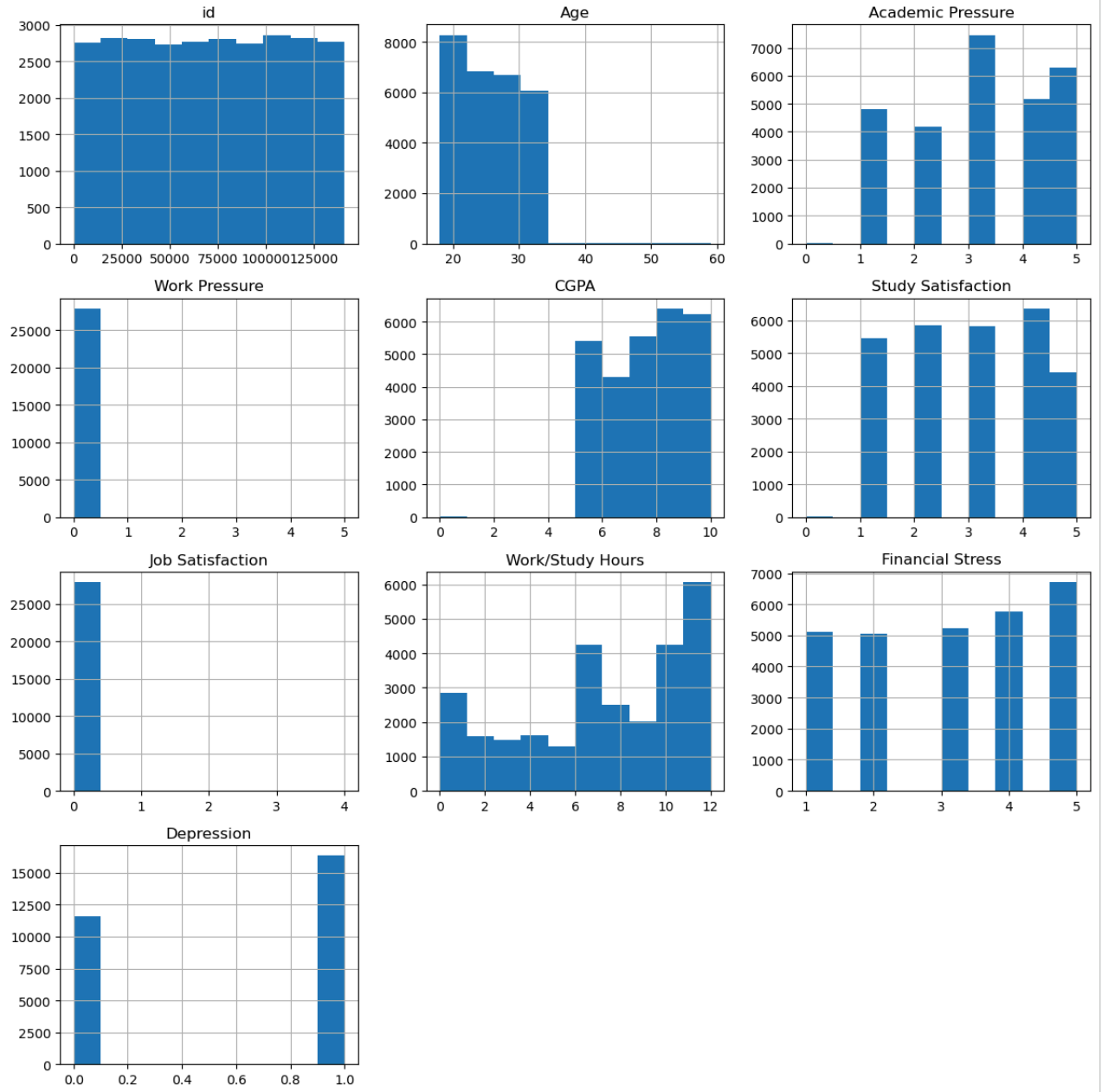



```
# Scatter plot: Age vs Work Pressure, colored by Depression
plt.scatter(x=df.Age[df.Depression == 0], y=df['Work Pressure'][df.Depression == 0])
plt.scatter(x=df.Age[df.Depression == 1], y=df['Work Pressure'][df.Depression == 1], c="orange")

plt.xlabel("Age")
plt.ylabel("Work Pressure")
plt.legend(['No Depression (0)', 'Depression (1)'])
plt.title("Work Pressure by Age and Depression Level")
plt.show()
```




```
# Plot histograms for all numeric columns
df.hist(figsize=(12, 12))
plt.tight_layout()
plt.show()
```




```
df["Financial Stress"].value_counts()
```

```
Financial Stress
5.0    6715
4.0    5775
3.0    5226
1.0    5121
2.0    5061
Name: count, dtype: int64
```

```
df["Academic Pressure"].value_counts()
```

```
Academic Pressure
3.0    7462
5.0    6296
4.0    5155
1.0    4801
2.0    4178
```

```
0.0       9
Name: count, dtype: int64
```

```
df["Work Pressure"].value_counts()
```

```
Work Pressure
0.0    27898
5.0        2
2.0        1
Name: count, dtype: int64
```

```
df["Sleep Duration"].value_counts()
```

```
Sleep Duration
Less than 5 hours    8310
7-8 hours            7346
5-6 hours            6183
More than 8 hours    6044
Others                 18
Name: count, dtype: int64
```

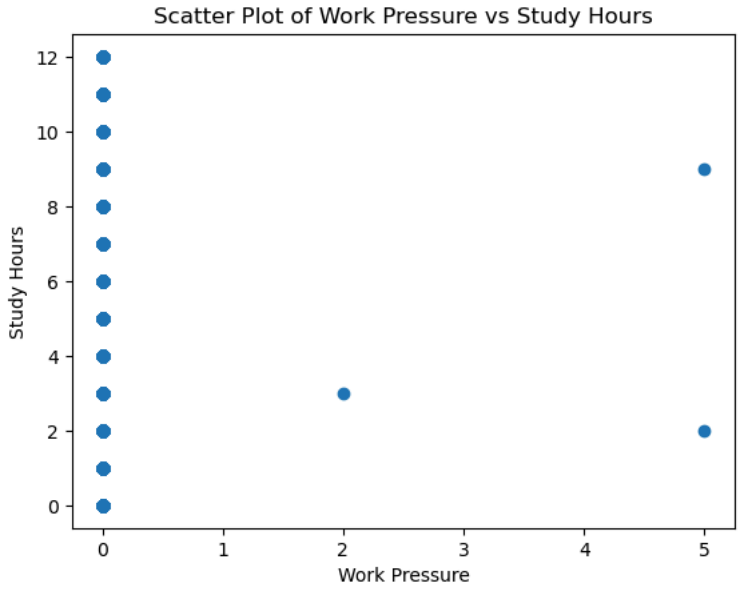
```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Simple scatter plot
plt.scatter(df['Work Pressure'], df['Work/Study Hours'])

# Labels and title
plt.xlabel('Work Pressure')
plt.ylabel('Study Hours')
plt.title('Scatter Plot of Work Pressure vs Study Hours')

# Show the plot
plt.show()
```



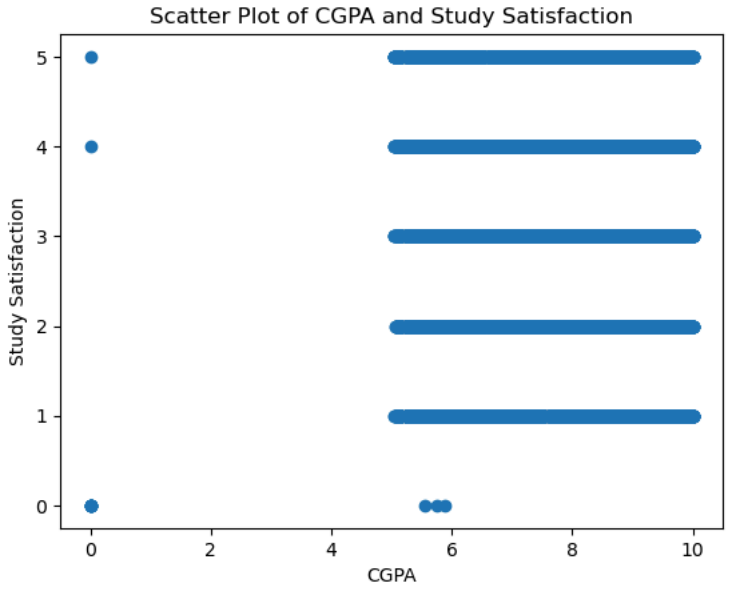

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Simple scatter plot
plt.scatter(df['CGPA'], df['Study Satisfaction'])

# Labels and title
plt.xlabel('CGPA')
plt.ylabel('Study Satisfaction')
plt.title('Scatter Plot of CGPA and Study Satisfaction')

# Show the plot
plt.show()
```



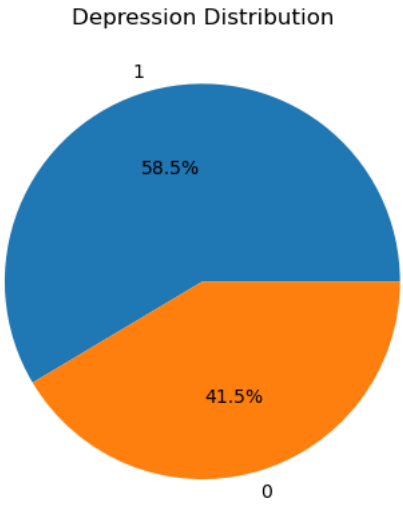
Start coding or generate with AI.

```
import pandas as pd
import matplotlib.pyplot as plt

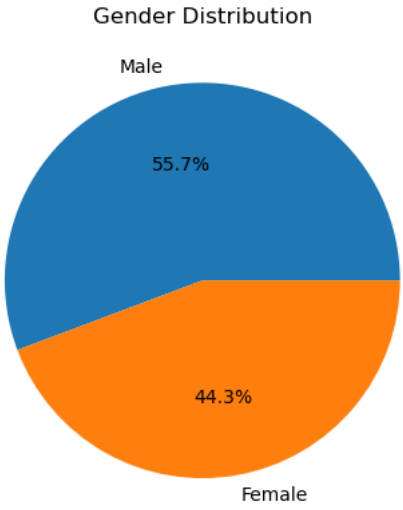
# Load the dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Count values in 'Depression' column
depression_counts = df['Depression'].value_counts()

# Create pie chart
plt.pie(depression_counts, labels=depression_counts.index, autopct='%1.1f%%')
plt.title('Depression Distribution')
plt.show()
```




```
gender_counts = df['Gender'].value_counts()
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%')
plt.title('Gender Distribution')
plt.show()
```

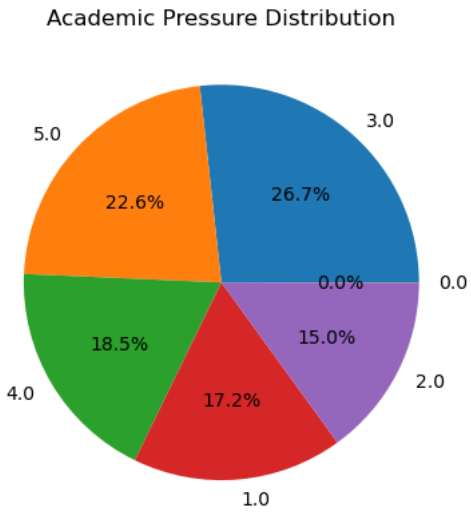



```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Count categories in 'Academic Pressure'
pressure_counts = df['Academic Pressure'].value_counts()

# Create pie chart
plt.pie(pressure_counts, labels=pressure_counts.index, autopct='%1.1f%%')
plt.title('Academic Pressure Distribution')
plt.show()
```




```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# Load the real dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Drop irrelevant columns (e.g., 'id', 'Profession')
df.drop(['id', 'Profession'], axis=1, inplace=True)

# Fill missing values if any
df.fillna(method='ffill', inplace=True)

# Encode categorical features
label_encoders = {}
categorical_columns = ['Gender', 'City', 'Sleep Duration', 'Dietary Habits',
                       'Degree', 'Have you ever had suicidal thoughts ?',
                       'Family History of Mental Illness']

for col in categorical_columns:
```

```
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Rename target column for consistency
df.rename(columns={'Depression': 'target'}, inplace=True)

# Display first few rows
df.head()
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_13728\3651391565.py:12: FutureWarning: DataFrame.fillna with 'method' is deprecat
  df.fillna(method='ffill', inplace=True)
```

	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours
0	1	33.0	51	5.0	0.0	8.97	2.0	0.0	0	0	3	1	3.0
1	0	24.0	3	2.0	0.0	5.90	5.0	0.0	0	1	10	0	3.0
2	1	31.0	44	3.0	0.0	7.03	5.0	0.0	2	0	5	0	9.0

```
df.head()
```

	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours
0	1	33.0	51	5.0	0.0	8.97	2.0	0.0	0	0	3	1	3.0
1	0	24.0	3	2.0	0.0	5.90	5.0	0.0	0	1	10	0	3.0
2	1	31.0	44	3.0	0.0	7.03	5.0	0.0	2	0	5	0	9.0

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

# Drop columns that are not useful for modeling
df.drop(['id', 'Profession'], axis=1, inplace=True)

# Handle missing values (you can change the method if needed)
df.fillna(method='ffill', inplace=True)

# List of categorical columns to encode
categorical_columns = [
    'Gender',
    'City',
    'Sleep Duration',
    'Dietary Habits',
    'Degree',
    'Have you ever had suicidal thoughts ?',
    'Family History of Mental Illness'
]

# Encode categorical columns
label_encoders = {}
for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Rename target column to 'target' for consistency
df.rename(columns={'Depression': 'target'}, inplace=True)

# Show the first few rows to verify
df.head()
```

```
C:\Users\USER\AppData\Local\Temp\ipykernel_13728\4087842162.py:12: FutureWarning: DataFrame.fillna with 'method' is deprecat
  df.fillna(method='ffill', inplace=True)
```

	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours
0	1	33.0	51	5.0	0.0	8.97	2.0	0.0	0	0	3	1	3.0
1	0	24.0	3	2.0	0.0	5.90	5.0	0.0	0	1	10	0	3.0
2	1	31.0	44	3.0	0.0	7.03	5.0	0.0	2	0	5	0	9.0

```
df.head()
```

	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Work/Study Hours
0	1	33.0	51	5.0	0.0	8.97	2.0	0.0	0	0	3	1	3.0
1	0	24.0	3	2.0	0.0	5.90	5.0	0.0	0	1	10	0	3.0
2	1	31.0	44	3.0	0.0	7.03	5.0	0.0	2	0	5	0	9.0

```
import pandas as pd

# Load dataset and clean column names
df = pd.read_csv("Student Depression Dataset.csv")
df.columns = df.columns.str.strip()

# Define actual categorical columns from your dataset
cat_cols = [
    'Gender',
    'City',
    'Sleep Duration',
    'Dietary Habits',
    'Degree',
    'Have you ever had suicidal thoughts ?',
    'Family History of Mental Illness'
]

# Define desired column order
target_col = 'Depression'
other_cols = [col for col in df.columns if col not in cat_cols + [target_col]]
new_order = cat_cols + other_cols + [target_col]

# Reorder the DataFrame
df = df[new_order]

df.head()
```

	Gender	City	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Family History of Mental Illness	id	Age	Profession	Academic Pressure	Work Pressure	CGPA	Sati
0	Male	Visakhapatnam	5-6 hours	Healthy	B.Pharm	Yes	No	2	33.0	Student	5.0	0.0	8.97	
1	Female	Bangalore	5-6 hours	Moderate	BSc	No	Yes	8	24.0	Student	2.0	0.0	5.90	
2	Male	Srinagar	Less than 5 hours	Healthy	BA	No	Yes	26	31.0	Student	3.0	0.0	7.03	

```
df.head()
```

	Gender	City	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?	Family History of Mental Illness	id	Age	Profession	Academic Pressure	Work Pressure	CGPA	Sati
0	Male	Visakhapatnam	5-6 hours	Healthy	B.Pharm	Yes	No	2	33.0	Student	5.0	0.0	8.97	
1	Female	Bangalore	5-6 hours	Moderate	BSc	No	Yes	8	24.0	Student	2.0	0.0	5.90	
2	Male	Srinagar	Less than 5 hours	Healthy	BA	No	Yes	26	31.0	Student	3.0	0.0	7.03	

```
# Check for missing values in the dataset
missing_summary = df.isnull().sum()

# Filter only columns that have missing values
missing_columns = missing_summary[missing_summary > 0]

# Print result
if not missing_columns.empty:
    print("After checking, the following columns have missing values:\n")
    print(missing_columns)
else:
    print("No columns with missing values were found in the Student Depression Dataset.")
```

```
After checking, the following columns have missing values:

Financial Stress    3
dtype: int64
```

```
import numpy as np

# Randomly introduce missing values into real columns
np.random.seed(42)  # For reproducibility

# Introduce 10% missing values in 'Sleep Duration'
df.loc[df.sample(frac=0.1).index, 'Sleep Duration'] = np.nan

# Introduce 20% missing values in 'Dietary Habits'
df.loc[df.sample(frac=0.2).index, 'Dietary Habits'] = np.nan

# Introduce 25% missing values in 'CGPA'
df.loc[df.sample(frac=0.25).index, 'CGPA'] = np.nan

# Introduce 40% missing values in 'Work Pressure'
df.loc[df.sample(frac=0.4).index, 'Work Pressure'] = np.nan

# Save the modified dataset to a new CSV file
df.to_csv("student_depression_with_missing.csv", index=False)

print("Missing values introduced and dataset saved as 'student_depression_with_missing.csv'.")
```

```
Missing values introduced and dataset saved as 'student_depression_with_missing.csv'.
```

```
df=pd.read_csv("Student Depression Dataset.csv")
df
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Diet Hab
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Hea
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moder
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Hea
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	7-8 hours	Moder
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	5-6 hours	Moder
...	...	...	...	...	...	...	...	...	...	...	...	
27896	140685	Female	27.0	Surat	Student	5.0	0.0	5.75	5.0	0.0	5-6 hours	Unhea
27897	140686	Male	27.0	Ludhiana	Student	2.0	0.0	9.40	3.0	0.0	Less than 5 hours	Hea
27898	140689	Male	31.0	Faridabad	Student	3.0	0.0	6.61	4.0	0.0	5-6 hours	Unhea
27899	140690	Female	18.0	Ludhiana	Student	5.0	0.0	6.88	2.0	0.0	Less than 5 hours	Hea
27900	140699	Male	27.0	Patna	Student	4.0	0.0	9.24	1.0	0.0	Less than	Hea

```
# Summary of missing values in each column
missing_summary = df.isnull().sum()

# Filter columns with missing values
missing_columns = missing_summary[missing_summary > 0]

# Conditional message
if not missing_columns.empty:
    print("After checking, the following columns have missing values:\n")
    print(missing_columns)
else:
    print("No columns with missing values were found.")
```

```
After checking, the following columns have missing values:

Financial Stress    3
dtype: int64
```

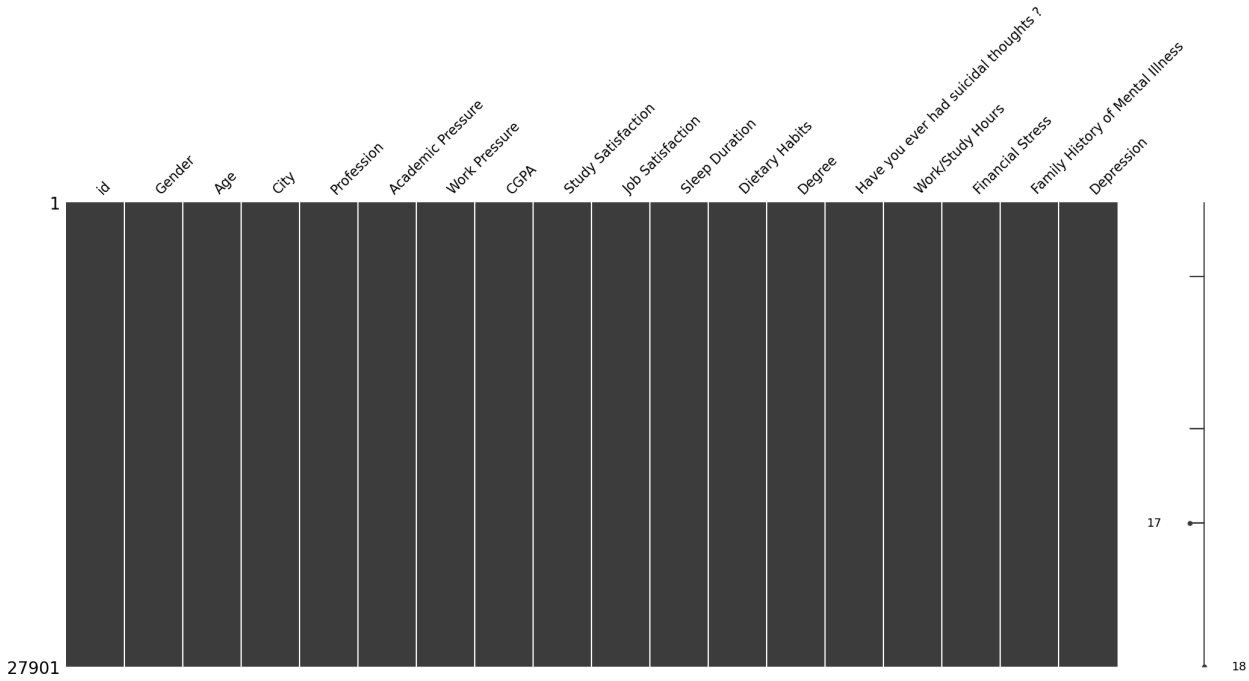
```
!pip install missingno
```

```
Requirement already satisfied: missingno in c:\users\user\anaconda3\lib\site-packages (0.5.2)
Requirement already satisfied: numpy in c:\users\user\anaconda3\lib\site-packages (from missingno) (1.26.4)
Requirement already satisfied: matplotlib in c:\users\user\anaconda3\lib\site-packages (from missingno) (3.9.2)
Requirement already satisfied: scipy in c:\users\user\anaconda3\lib\site-packages (from missingno) (1.13.1)
Requirement already satisfied: seaborn in c:\users\user\anaconda3\lib\site-packages (from missingno) (0.13.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (1
Requirement already satisfied: cycler>=0.10 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (0.11.
Requirement already satisfied: fonttools>=4.22.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (
Requirement already satisfied: packaging>=20.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (24
Requirement already satisfied: pillow>=8 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno) (3
Requirement already satisfied: python-dateutil>=2.7 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->missingno
Requirement already satisfied: pandas>=1.2 in c:\users\user\anaconda3\lib\site-packages (from seaborn->missingno) (2.2.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\user\anaconda3\lib\site-packages (from pandas>=1.2->seaborn->missing
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\lib\site-packages (from pandas>=1.2->seaborn->missi
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib-
```

```
import missingno as msno

# Visualize missing data as a matrix
msno.matrix(df)
```

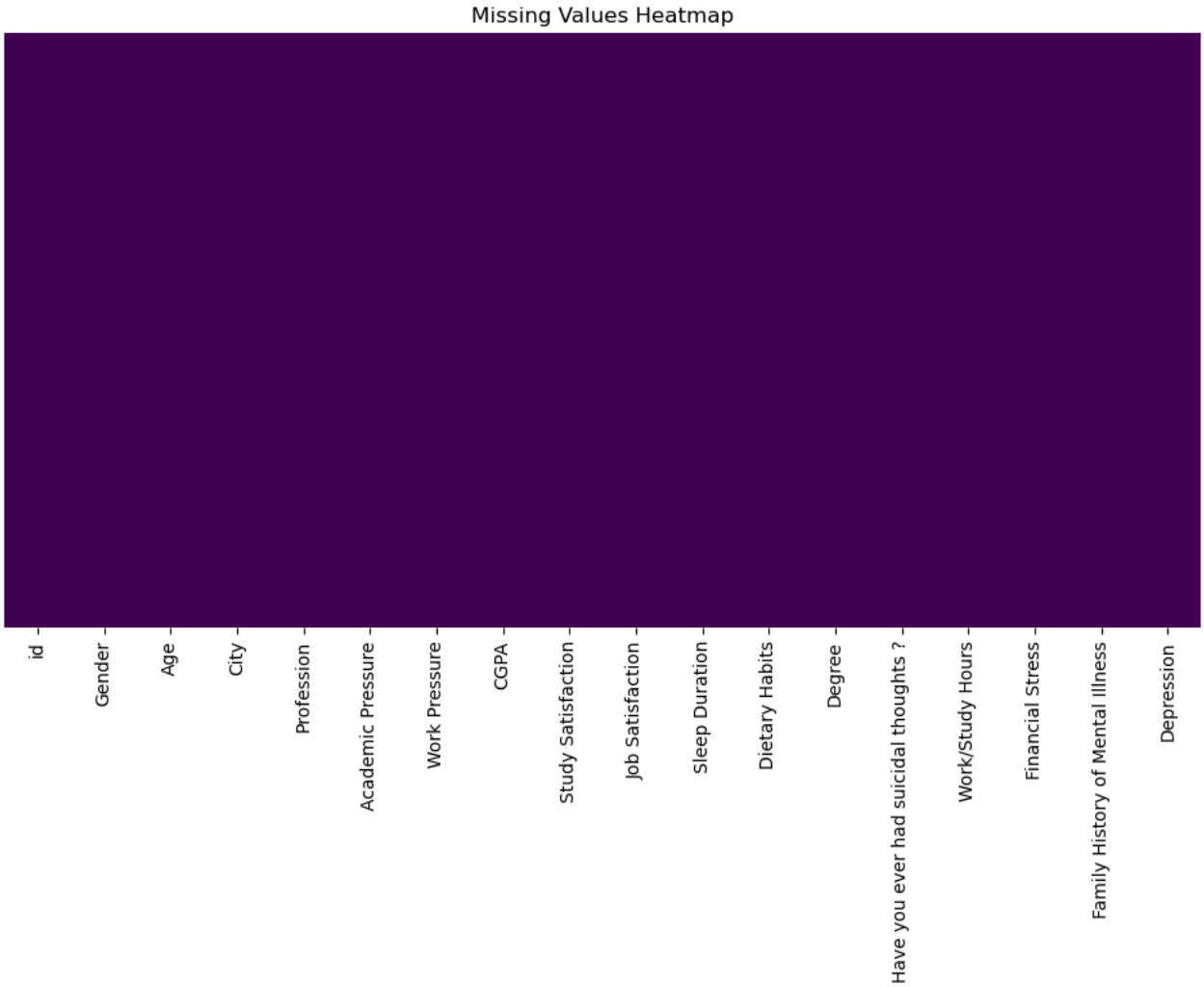
<Axes: >




```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a boolean mask for missing values
missing_data = df.isnull()

# Plot heatmap
plt.figure(figsize=(12, 6))
sns.heatmap(missing_data, cbar=False, cmap='viridis', yticklabels=False)
plt.title("Missing Values Heatmap")
plt.show()
```




```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.utils import resample
from collections import Counter
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Generate synthetic imbalanced dataset
X, y = make_classification(n_samples=1000, n_features=20,
                           n_informative=10, n_redundant=5,
                           n_classes=2, class_sep=2,
                           weights=[0.9, 0.1], random_state=42)

# Step 2: Create DataFrame
feature_names = [f"feature_{i}" for i in range(X.shape[1])]
df_imbalanced = pd.DataFrame(X, columns=feature_names)
df_imbalanced["target"] = y

# Step 3: Split into train and test
X = df_imbalanced.drop(columns="target")
y = df_imbalanced["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Step 4: Separate classes
X_majority = X_train[y_train == 0]
X_minority = X_train[y_train == 1]
y_majority = y_train[y_train == 0]
y_minority = y_train[y_train == 1]

# Step 5: Oversample minority class
X_minority_oversampled, y_minority_oversampled = resample(
    X_minority, y_minority,
    replace=True,
    n_samples=len(X_majority),
    random_state=42
)

# Step 6: Combine majority with oversampled minority
```

```
X_train_oversampled = pd.concat([X_majority, X_minority_oversampled])
y_train_oversampled = pd.concat([y_majority, y_minority_oversampled])

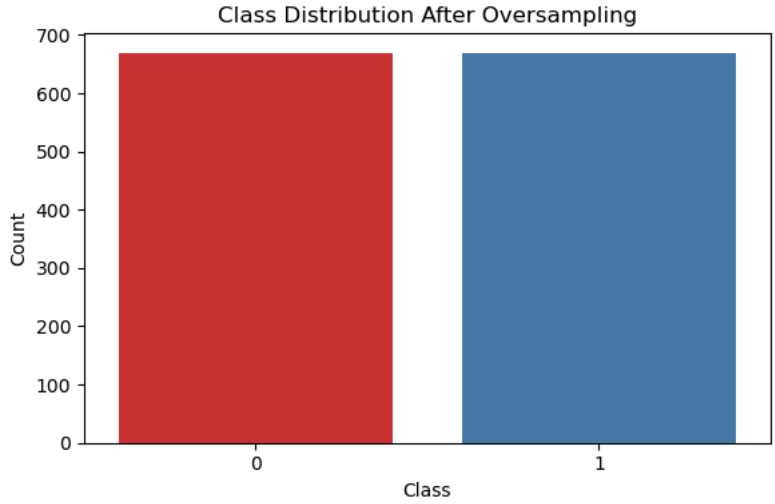
# Step 7: Print class distributions
print("Original class distribution:", Counter(y_train))
print("Resampled class distribution:", Counter(y_train_oversampled))

# Optional: Visualize the new balanced distribution
plt.figure(figsize=(6, 4))
sns.countplot(x=y_train_oversampled, palette="Set1")
plt.title("Class Distribution After Oversampling")
plt.xlabel("Class")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

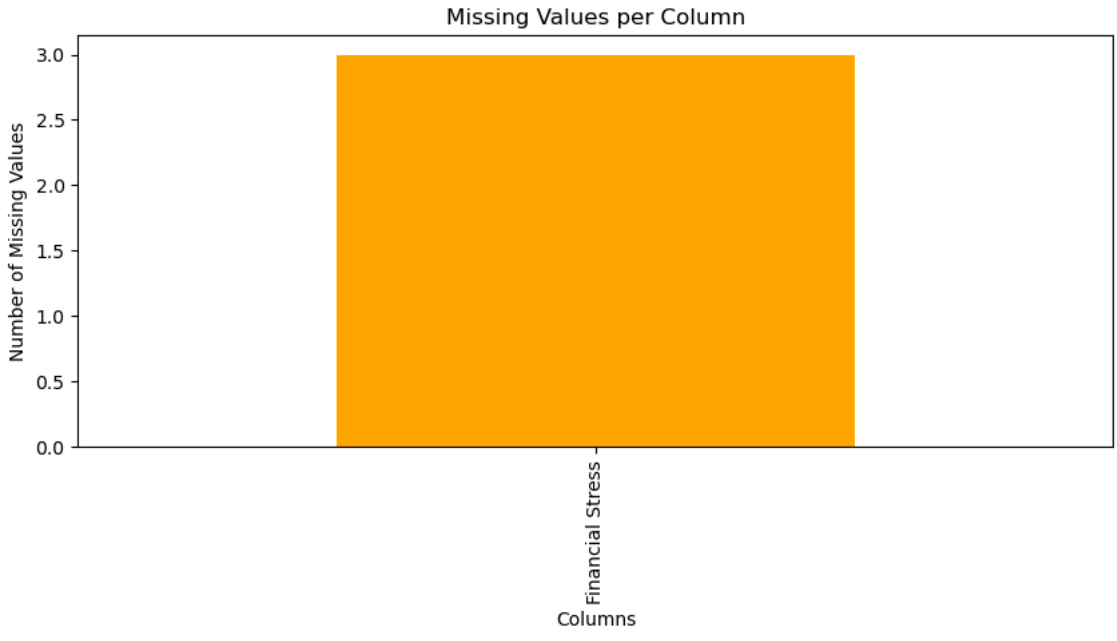
```
Original class distribution: Counter({0: 669, 1: 81})
Resampled class distribution: Counter({0: 669, 1: 669})
C:\Users\USER\AppData\Local\Temp\ipykernel_13728\3852249781.py:49: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(x=y_train_oversampled, palette="Set1")
```



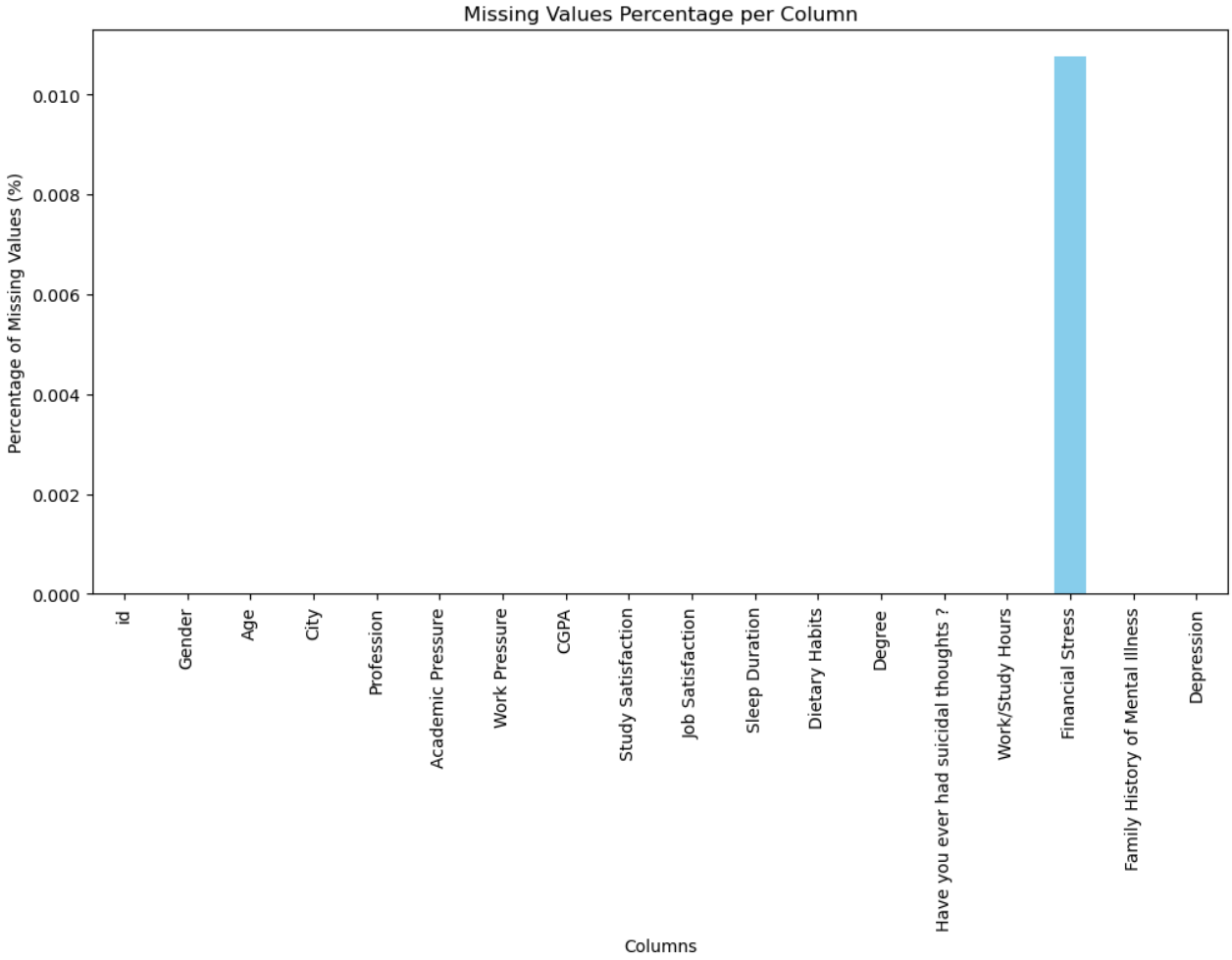

```
# Bar plot of missing values per column
missing_counts = df.isnull().sum()
plt.figure(figsize=(10, 4))
missing_counts[missing_counts > 0].plot(kind='bar', color='orange')
plt.title("Missing Values per Column")
plt.ylabel("Number of Missing Values")
plt.xlabel("Columns")
plt.show()
```



```
import matplotlib.pyplot as plt

# Plot percentage of missing values per column
missing_percentage = df.isnull().mean() * 100

plt.figure(figsize=(12, 6))
missing_percentage.plot(kind='bar', color='skyblue')
plt.title("Missing Values Percentage per Column")
plt.ylabel("Percentage of Missing Values (%)")
plt.xlabel("Columns")
plt.show()
```




```
# Fill missing numerical values safely
for col in ['Financial Stress', 'Age', 'CGPA']:
    if col in df.columns and df[col].isnull().any():
        df[col] = pd.to_numeric(df[col], errors='coerce')  # ensure numeric
        df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values safely
for col in ['Dietary Habits', 'Sleep Duration', 'City']:
    if col in df.columns and df[col].isnull().any():
        df[col] = df[col].fillna(method='ffill')
```

```
# Summary of missing values in each column
missing_summary = df.isnull().sum()

# Filter columns with missing values
missing_columns = missing_summary[missing_summary > 0]

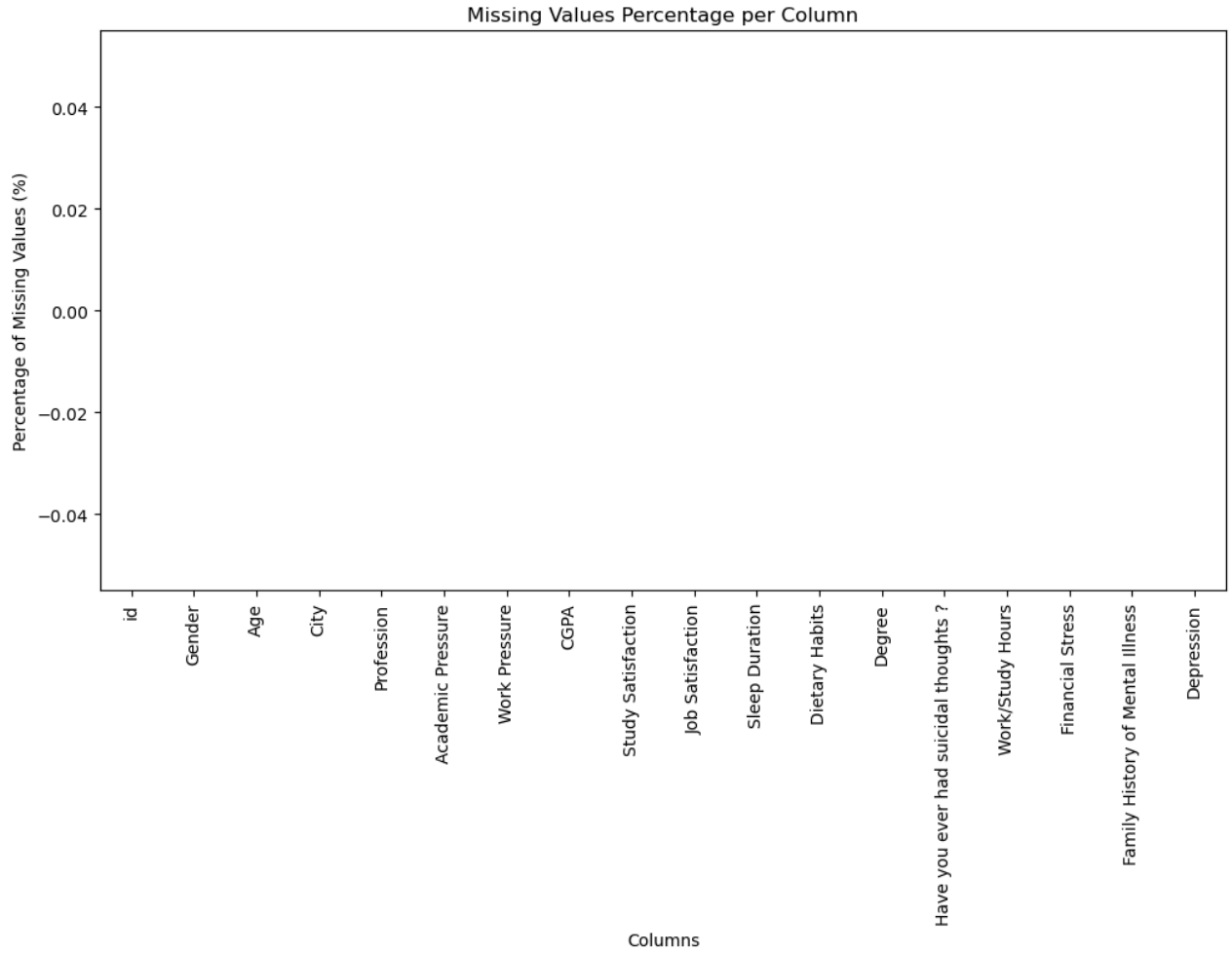
# Conditional message
if not missing_columns.empty:
    print("After checking, the following columns have missing values:\n")
    print(missing_columns)
else:
    print("No columns with missing values were found.")
```

```
No columns with missing values were found.
```

```
import matplotlib.pyplot as plt

# Plot percentage of missing values per column
missing_percentage = df.isnull().mean() * 100

plt.figure(figsize=(12, 6))
missing_percentage.plot(kind='bar', color='skyblue')
plt.title("Missing Values Percentage per Column")
plt.ylabel("Percentage of Missing Values (%)")
plt.xlabel("Columns")
plt.show()
```




```
from sklearn.preprocessing import LabelEncoder

# Initialize dictionary to store encoders (optional, useful for inverse transforming later)
label_encoders = {}

# List of actual categorical columns in the dataset
categorical_cols = [
    'Gender',
    'City',
    'Sleep Duration',
    'Dietary Habits',
```

```
    'Degree',
    'Have you ever had suicidal thoughts ?',
    'Family History of Mental Illness',
    'Profession'
]

# Apply Label Encoding to each categorical column
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le  # store the encoder

# Show the updated dataframe
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have y ever h suicid though
0	2	1	33.0	51	11	5.0	0.0	8.97	2.0	0.0	0	0	3	
1	8	0	24.0	3	11	2.0	0.0	5.90	5.0	0.0	0	1	10	
2	26	1	31.0	44	11	3.0	0.0	7.03	5.0	0.0	2	0	5	

```
# One-hot encode categorical features using get_dummies
df_encoded = pd.get_dummies(df, columns=[
    'Gender',
    'City',
    'Sleep Duration',
    'Dietary Habits',
    'Degree',
    'Have you ever had suicidal thoughts ?',
    'Family History of Mental Illness',
    'Profession'
], drop_first=True)

# Display the encoded dataframe
df_encoded.head()
```

	id	Age	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Work/Study Hours	Financial Stress	Depression	...	Profession_4	Profe
0	2	33.0	5.0	0.0	8.97	2.0	0.0	3.0	1.0	1	...	False	
1	8	24.0	2.0	0.0	5.90	5.0	0.0	3.0	2.0	0	...	False	
2	26	31.0	3.0	0.0	7.03	5.0	0.0	9.0	1.0	0	...	False	
3	30	28.0	3.0	0.0	5.59	2.0	0.0	4.0	5.0	1	...	False	
4	32	25.0	4.0	0.0	8.13	3.0	0.0	1.0	1.0	0	...	False	

5 rows × 111 columns

```
import matplotlib.pyplot as plt

# Plot histograms for all numeric columns
df.select_dtypes(include=[np.number]).hist(bins=30,color="pink",figsize=(16, 14))

plt.suptitle("Histograms of Numeric Columns")

plt.show()
```




```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

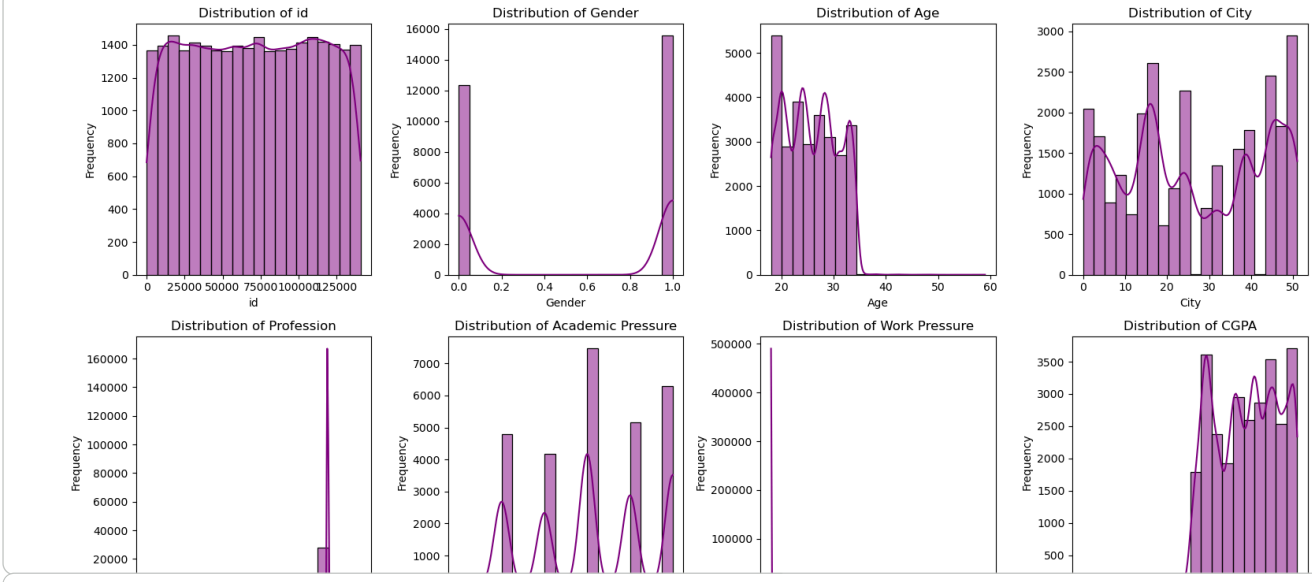
# Identify numeric columns
numeric_cols = df.select_dtypes(include=[np.number]).columns
num_cols = len(numeric_cols)

# Define number of columns in the grid
ncols = 4  # Adjust depending on how many plots you want per row
nrows = int(np.ceil(num_cols / ncols))  # Calculate required number of rows

# Create subplots dynamically based on the number of numeric columns
plt.figure(figsize=(ncols * 4, nrows * 4))  # Increase size for clarity
```

```
# Loop through numeric columns and plot each one
for i, col in enumerate(numeric_cols):
    plt.subplot(nrows, ncols, i + 1)
    sns.histplot(df[col], kde=True, bins=20, color='purple')
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```




```
from scipy.stats import zscore
import pandas as pd
import numpy as np

# Select only numeric columns from the dataset
numeric_df = df.select_dtypes(include=[np.number])

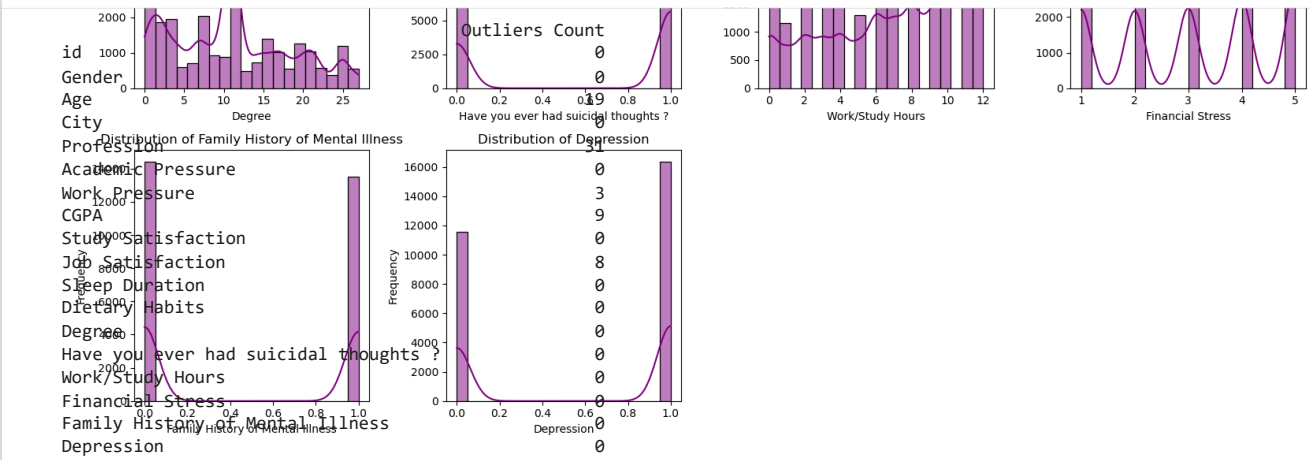
# Calculate Z-scores for all numeric features
z_scores = zscore(numeric_df)

# Identify outliers (Z-score > 3 or < -3)
outliers = np.abs(z_scores) > 3

# Count outliers in each column
outliers_count = np.sum(outliers, axis=0)

# Create a DataFrame to show outlier counts per column
outliers_count_df = pd.DataFrame(outliers_count, index=numeric_df.columns, columns=["Outliers Count"])

# Display outlier counts
print(outliers_count_df)
```




```
                                   Outliers Count
id                                             0
Gender                                         0
Age                                            19
City                                           0
Profession                                     31
Academic Pressure                              0
Work Pressure                                  3
CGPA                                           9
Study Satisfaction                             0
Job Satisfaction                               8
Sleep Duration                                 0
Dietary Habits                                 0
Degree                                         0
Have you ever had suicidal thoughts ?          0
Work/Study Hours                               0
Financial Stress                               0
Family History of Mental Illness              0
Depression                                     0
```

```
# Select numeric columns
numeric_df = df.select_dtypes(include=[np.number])

# Calculate Q1 and Q3
Q1 = numeric_df.quantile(0.25)
Q3 = numeric_df.quantile(0.75)
IQR = Q3 - Q1

# Identify outliers using the IQR method
outliers_iqr = (numeric_df < (Q1 - 1.5 * IQR)) | (numeric_df > (Q3 + 1.5 * IQR))

# Count outliers in each numeric column
outliers_count_iqr = outliers_iqr.sum()

# Create a DataFrame to display outlier counts
outliers_count_iqr_df = pd.DataFrame(outliers_count_iqr, columns=["Outliers Count"])
```

```
# Display the result
print(outliers_count_iqr_df)
```

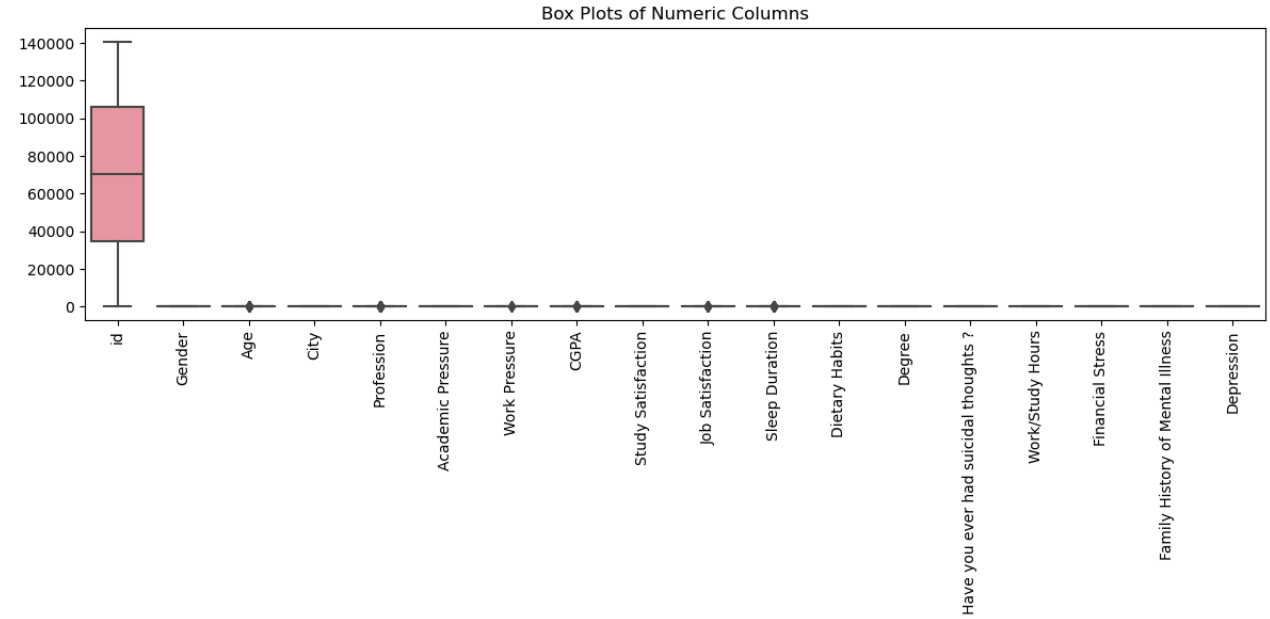
```
                                     Outliers Count
id                                                0
Gender                                            0
Age                                              12
City                                              0
Profession                                       31
Academic Pressure                                 0
Work Pressure                                     3
CGPA                                              9
Study Satisfaction                                0
Job Satisfaction                                  8
Sleep Duration                                   18
Dietary Habits                                    0
Degree                                            0
Have you ever had suicidal thoughts ?             0
Work/Study Hours                                  0
Financial Stress                                  0
Family History of Mental Illness                  0
Depression                                        0
```

```
import matplotlib.pyplot as plt

# Replace with your actual DataFrame loading or creation
# df = pd.read_csv("your_dataset.csv")

# Select only numeric columns
numeric_cols = df.select_dtypes(include=[np.number])

# Plot boxplots for each numeric column
plt.figure(figsize=(12, 6))
sns.boxplot(data=numeric_cols)
plt.title("Box Plots of Numeric Columns")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```




```
import numpy as np
from scipy.stats import zscore

# Select numeric columns
numeric_cols = df.select_dtypes(include=[np.number]).columns

# Compute Z-scores
z_scores = df[numeric_cols].apply(zscore)

# Identify outliers as values with |Z| > 3
outliers = (np.abs(z_scores) > 3)
```

```
# Replace outliers with NaN
df[numeric_cols] = df[numeric_cols].mask(outliers)

# Fill NaNs with column means|
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
```

```
import pandas as pd
import numpy as np
from scipy.stats import zscore

# Select only numeric columns
numeric_df = df.select_dtypes(include=[np.number])

# Calculate Z-scores for numeric features
z_scores = numeric_df.apply(zscore)

# Identify outliers (Z-score > 3 or < -3)
outliers = np.abs(z_scores) > 3

# Count outliers in each column
outliers_count = outliers.sum()

# Create and display a DataFrame with outlier counts
outliers_count_df = pd.DataFrame({
    "Outliers Count": outliers_count
})

print(outliers_count_df)
```

```
                                       Outliers Count
id                                                  0
Gender                                              0
Age                                                 0
City                                                0
Profession                                          0
Academic Pressure                                   0
Work Pressure                                       0
CGPA                                                0
Study Satisfaction                                  0
Job Satisfaction                                    0
Sleep Duration                                      0
Dietary Habits                                      0
Degree                                              0
Have you ever had suicidal thoughts ?               0
Work/Study Hours                                    0
Financial Stress                                    0
Family History of Mental Illness                    0
Depression                                          0
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler, QuantileTransformer

# Exclude non-numeric and target columns
excluded_columns = ['id', 'Gender', 'City', 'Profession', 'Degree',
                    'Dietary Habits', 'Sleep Duration',
                    'Have you ever had suicidal thoughts ?',
                    'Family History of Mental Illness', 'Depression']

# Identify numeric features
numeric_features = df.select_dtypes(include=['number']).drop(columns=['id', 'Depression']).columns.tolist()

# Scalers to apply
scalers = {
    "StandardScaler": StandardScaler(),
    "MinMaxScaler": MinMaxScaler(),
    "RobustScaler": RobustScaler(),
    "QuantileTransformer": QuantileTransformer(output_distribution='normal')
}

# Dictionary to hold scaled DataFrames
scaled_data = {}
for name, scaler in scalers.items():
    scaled = scaler.fit_transform(df[numeric_features])
    scaled_data[name] = pd.DataFrame(scaled, columns=numeric_features)

# Example: View the first few rows of the StandardScaler result
scaled_data["StandardScaler"].head()
```

	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	
0	0.891416	1.476662	1.510522	0.0	1.345543	0.0	0.895638	-0.693425	0.0	-1.421696	-1.182363	-
1	-1.121811	-0.371102	-1.424227	0.0	-0.826104	0.0	-1.201018	1.510636	0.0	-1.421696	-0.375705	-
2	0.891416	1.066048	1.082538	0.0	-0.102222	0.0	-0.429285	1.510636	0.0	0.459582	-1.182363	-

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Step 1: Define columns to exclude
excluded_columns = ['id', 'Gender', 'City', 'Profession', 'Degree',
                    'Dietary Habits', 'Sleep Duration',
                    'Have you ever had suicidal thoughts ?',
                    'Family History of Mental Illness', 'Depression']

# Step 2: Select numeric features for PCA
numeric_features = df.select_dtypes(include=['number']).drop(columns=['id', 'Depression']).columns.tolist()
X = df[numeric_features]

# Step 3: Standardize the numeric data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Apply PCA
pca = PCA(n_components=5)
pca_components = pca.fit_transform(X_scaled)

# Step 5: Create a DataFrame with PCA components
pca_df = pd.DataFrame(pca_components, columns=[f"PCA_{i}" for i in range(1, 6)])

# View the result
pca_df.head()
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Step 1: Define columns to exclude
excluded_columns = ['id', 'Gender', 'City', 'Profession', 'Degree',
                    'Dietary Habits', 'Sleep Duration',
                    'Have you ever had suicidal thoughts ?',
                    'Family History of Mental Illness', 'Depression']

# Step 2: Select numeric features for PCA
numeric_features = df.select_dtypes(include=['number']).drop(columns=['id', 'Depression']).columns.tolist()
X = df[numeric_features]

# Step 3: Standardize the numeric data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Apply PCA
pca = PCA(n_components=5)
pca_components = pca.fit_transform(X_scaled)

# Step 5: Create a DataFrame with PCA components
pca_df = pd.DataFrame(pca_components, columns=[f"PCA_{i}" for i in range(1, 6)])

# View the result
pca_df.head()
```

	PCA_1	PCA_2	PCA_3	PCA_4	PCA_5
0	-0.480517	-0.306306	1.251014	-1.795344	0.933928
1	-1.938866	-0.342164	-1.690254	-0.239603	-0.364883
2	-2.188318	-0.588600	-0.297472	0.097601	0.626260
3	0.617582	-1.962417	-1.027950	-0.232059	-0.064495
4	-0.495181	-0.361199	-0.012725	-0.815654	-0.404911

```
# Access PCA component weights (loadings)
loadings = pd.DataFrame(
    pca.components_,
    columns=numeric_features,  # Only numeric features used in PCA
    index=[f"PCA_{i}" for i in range(1, 6)]
```

```
)

# Show features contributing most to the first principal component
top_contributors_pca1 = loadings.T.sort_values(by="PCA_1", ascending=False).head()

top_contributors_pca1
```

	PCA_1	PCA_2	PCA_3	PCA_4	PCA_5
Have you ever had suicidal thoughts ?	0.532203	-0.102124	0.017652	0.018004	0.021466
Academic Pressure	0.485823	-0.167968	0.004498	0.025869	-0.123368
Financial Stress	0.433588	-0.040836	-0.025179	0.067323	0.024636
Dietary Habits	0.276599	0.245518	0.069323	0.405540	0.297288
Work/Study Hours	0.273634	-0.150780	0.138290	-0.197822	0.242537

```
from sklearn.feature_selection import SelectKBest, f_classif

# Step 1: Exclude categorical/non-numeric and target columns
excluded_columns = ['id', 'Gender', 'City', 'Profession', 'Degree',
                    'Dietary Habits', 'Sleep Duration',
                    'Have you ever had suicidal thoughts ?',
                    'Family History of Mental Illness', 'Depression']

# Step 2: Define input features (X) and target (y)
numeric_features = df.select_dtypes(include=['number']).drop(columns=['id', 'Depression']).columns.tolist()
X = df[numeric_features]
y = df['Depression']

# Step 3: Apply SelectKBest
selector = SelectKBest(score_func=f_classif, k=10)
X_new = selector.fit_transform(X, y)

# Step 4: Get selected feature names
selected_indices = selector.get_support(indices=True)
selected_features = [numeric_features[i] for i in selected_indices]
selected_df = df[selected_features]

# Display selected features
selected_df.head()
```

```
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\feature_selection\_univariate_selection.py:112: UserWarning: Features [3 5
  warnings.warn("Features %s are constant." % constant_features_idx, UserWarning)
C:\Users\USER\anaconda3\Lib\site-packages\sklearn\feature_selection\_univariate_selection.py:113: RuntimeWarning: invalid va
  f = msb / msw
```

	Age	City	Academic Pressure	Study Satisfaction	Sleep Duration	Dietary Habits	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress	Family History of Mental Illness
0	33.0	51	5.0	2.0	0	0	1	3.0	1.0	0
1	24.0	3	2.0	5.0	0	1	0	3.0	2.0	1
2	31.0	44	3.0	5.0	2	0	0	9.0	1.0	1

```
import seaborn as sns
import matplotlib.pyplot as plt

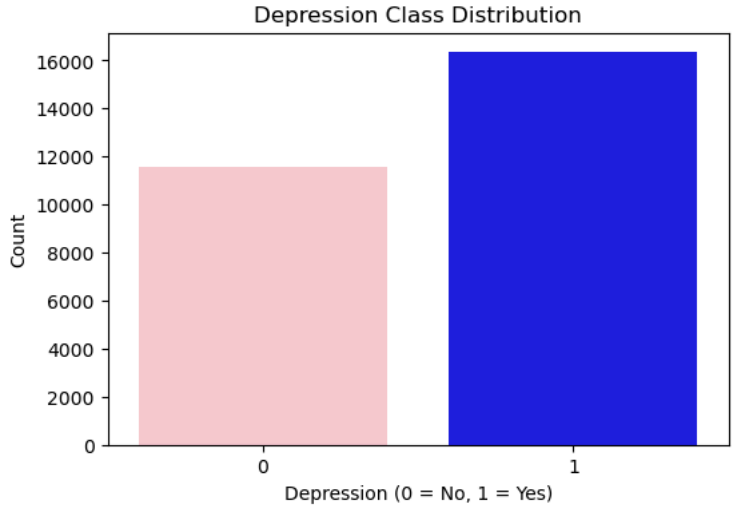
# Count the class distribution in the actual dataset
class_counts = df["Depression"].value_counts()
print("Class Distribution in Dataset:\n", class_counts)

# Visualize the class imbalance with a custom color palette
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Depression", palette=["PINK", "BLUE"])  # Custom blue & orange
plt.title("Depression Class Distribution")
plt.xlabel("Depression (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```

```
Class Distribution in Dataset:
 Depression
1    16336
0    11565
Name: count, dtype: int64
C:\Users\USER\AppData\Local\Temp\ipykernel_13728\196928471.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(data=df, x="Depression", palette=["PINK", "BLUE"])  # Custom blue & orange
```




```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.utils import resample
from collections import Counter
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Generate synthetic imbalanced dataset
X, y = make_classification(n_samples=1000, n_features=20,
                           n_informative=10, n_redundant=5,
                           n_classes=2, class_sep=2,
                           weights=[0.9, 0.1], random_state=42)

# Step 2: Create DataFrame
feature_names = [f"feature_{i}" for i in range(X.shape[1])]
df_imbalanced = pd.DataFrame(X, columns=feature_names)
df_imbalanced["target"] = y

# Step 3: Split into train and test
X = df_imbalanced.drop(columns="target")
y = df_imbalanced["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Step 4: Separate classes
X_majority = X_train[y_train == 0]
X_minority = X_train[y_train == 1]
y_majority = y_train[y_train == 0]
y_minority = y_train[y_train == 1]

# Step 5: Oversample minority class
X_minority_oversampled, y_minority_oversampled = resample(
    X_minority, y_minority,
    replace=True,
    n_samples=len(X_majority),
    random_state=42
)

# Step 6: Combine majority with oversampled minority
X_train_oversampled = pd.concat([X_majority, X_minority_oversampled])
y_train_oversampled = pd.concat([y_majority, y_minority_oversampled])

# Step 7: Print class distributions
print("Original class distribution:", Counter(y_train))
print("Resampled class distribution:", Counter(y_train_oversampled))

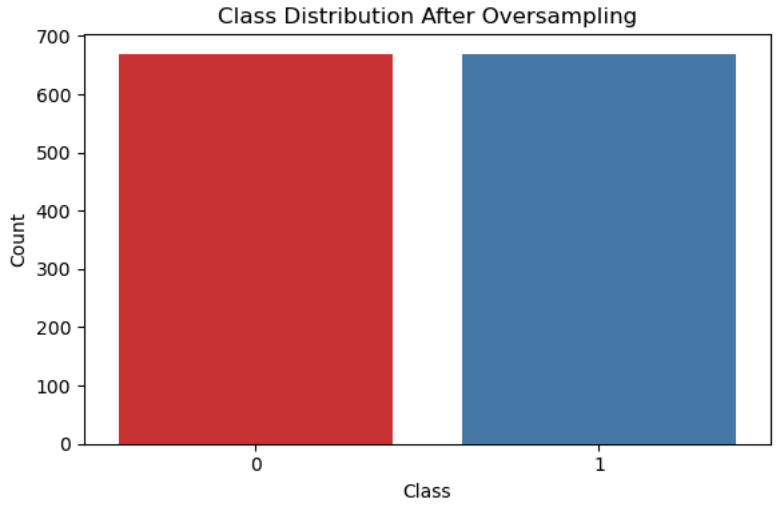
# Optional: Visualize the new balanced distribution
plt.figure(figsize=(6, 4))
sns.countplot(x=y_train_oversampled, palette="Set1")
plt.title("Class Distribution After Oversampling")
```

```
plt.xlabel("Class")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
Original class distribution: Counter({0: 669, 1: 81})
Resampled class distribution: Counter({0: 669, 1: 669})
C:\Users\USER\AppData\Local\Temp\ipykernel_13728\3298818959.py:49: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(x=y_train_oversampled, palette="Set1")
```




```
df_imbalanced.head()
```

	feature_0	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	feature_9	...	feature
0	2.634005	-0.909322	-1.786953	0.172431	0.315743	1.065964	0.017832	-2.542546	-1.513315	-0.765207	...	1.857
1	2.789649	0.088372	-1.230989	1.214075	-4.071693	-0.497370	0.714550	-0.933563	-4.875796	-0.992287	...	1.694
2	-1.355643	1.218601	-2.603269	-0.126606	-1.542429	0.717264	-0.859399	-2.042190	-2.243134	-0.019769	...	1.544
3	-2.596198	0.142573	2.772434	0.360639	2.739551	-0.438525	0.281949	1.345145	1.991731	-0.610865	...	-2.025
4	3.840049	-3.489600	-1.844902	-2.594362	-6.632428	0.459637	0.913607	-2.143505	1.196769	-3.040928	...	5.176

5 rows × 21 columns

```
### 2. **Over-Sampling**

##Over-sampling involves increasing the number of instances in the minority class by duplicating existing samples. This car

#### Pros:
## Helps in balancing the dataset.
##Can improve model performance for the minority class.

#### Cons:
##Potential risk of overfitting.
##Does not introduce new data, merely replicates existing data.
```

```
from sklearn.utils import resample
from collections import Counter
from sklearn.model_selection import train_test_split
# Separate features and target
X = df_imbalanced.drop(columns="target")
y = df_imbalanced["target"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Separate the minority and majority classes
X_majority = X_train[y_train == 0]  # Assuming 0 is the majority class
X_minority = X_train[y_train == 1]  # Assuming 1 is the minority class

y_majority = y_train[y_train == 0]
y_minority = y_train[y_train == 1]

# Oversample the minority class
X_minority_oversampled, y_minority_oversampled = resample(X_minority,
    y_minority,
```

```
    replace=True,       # Sample with replacement
    n_samples=len(X_majority),  # Match the number of majority class
    random_state=42)

# Combine majority class with oversampled minority class
X_train_oversampled = pd.concat([X_majority, X_minority_oversampled])
y_train_oversampled = pd.concat([y_majority, y_minority_oversampled])

# Print class distribution
print("Original class distribution:", Counter(y_train))
print("Resampled class distribution:", Counter(y_train_oversampled))
```

```
Original class distribution: Counter({0: 669, 1: 81})
Resampled class distribution: Counter({0: 669, 1: 669})
```

```
import pandas as pd
from sklearn.utils import resample
from collections import Counter
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

# Drop non-feature columns
df = df.drop(columns=["id", "City", "Profession", "Degree"])

# Convert categorical features to numerical using one-hot encoding
df = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df.drop(columns="Depression")
y = df["Depression"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)

# Separate the majority and minority classes
X_majority = X_train[y_train == 0]
X_minority = X_train[y_train == 1]

y_majority = y_train[y_train == 0]
y_minority = y_train[y_train == 1]

# Oversample the minority class
X_minority_oversampled, y_minority_oversampled = resample(
    X_minority,
    y_minority,
    replace=True,
    n_samples=len(X_majority),
    random_state=42
)

# Combine majority and oversampled minority
X_train_oversampled = pd.concat([X_majority, X_minority_oversampled])
y_train_oversampled = pd.concat([y_majority, y_minority_oversampled])

# Print class distributions
print("Original class distribution:", Counter(y_train))
print("Resampled class distribution:", Counter(y_train_oversampled))
```

```
Original class distribution: Counter({1: 12252, 0: 8673})
Resampled class distribution: Counter({0: 8673, 1: 8673})
```

Feature enginnering

```
import pandas as pd

# Load the actual dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Display the first 5 rows to confirm it's loaded
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Deg
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Ph
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	

```
# Drop the unnamed index column
df.drop(columns=['id'], inplace=True)
```

```
# Display shape and column info
print("Shape:", df.shape)
```

```
Shape: (27901, 17)
```

```
print("\nColumn Info:")
df.info()
```

```
Column Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27901 entries, 0 to 27900
Data columns (total 17 columns):
 #   Column                                 Non-Null Count  Dtype
---  ------                                 --------------  -----
 0   Gender                                 27901 non-null  object
 1   Age                                    27901 non-null  float64
 2   City                                   27901 non-null  object
 3   Profession                             27901 non-null  object
 4   Academic Pressure                      27901 non-null  float64
 5   Work Pressure                          27901 non-null  float64
 6   CGPA                                   27901 non-null  float64
 7   Study Satisfaction                     27901 non-null  float64
 8   Job Satisfaction                       27901 non-null  float64
 9   Sleep Duration                         27901 non-null  object
 10  Dietary Habits                         27901 non-null  object
 11  Degree                                 27901 non-null  object
 12  Have you ever had suicidal thoughts ?  27901 non-null  object
 13  Work/Study Hours                       27901 non-null  float64
 14  Financial Stress                       27898 non-null  float64
 15  Family History of Mental Illness       27901 non-null  object
 16  Depression                             27901 non-null  int64
dtypes: float64(8), int64(1), object(8)
memory usage: 3.6+ MB
```

```
# Check for null values
print("\nMissing Values:\n", df.isnull().sum())
```

```
Missing Values:
 Gender                                 0
Age                                    0
City                                   0
Profession                             0
Academic Pressure                      0
Work Pressure                          0
CGPA                                   0
Study Satisfaction                     0
Job Satisfaction                       0
Sleep Duration                         0
Dietary Habits                         0
Degree                                 0
Have you ever had suicidal thoughts ?  0
Work/Study Hours                       0
Financial Stress                       3
Family History of Mental Illness       0
Depression                             0
dtype: int64
```

```
# Check unique values in 'Gender' and 'Profession'
print("\nGender Counts:\n", df['Gender'].value_counts())
print("\nProfession Counts:\n", df['Profession'].value_counts())
```

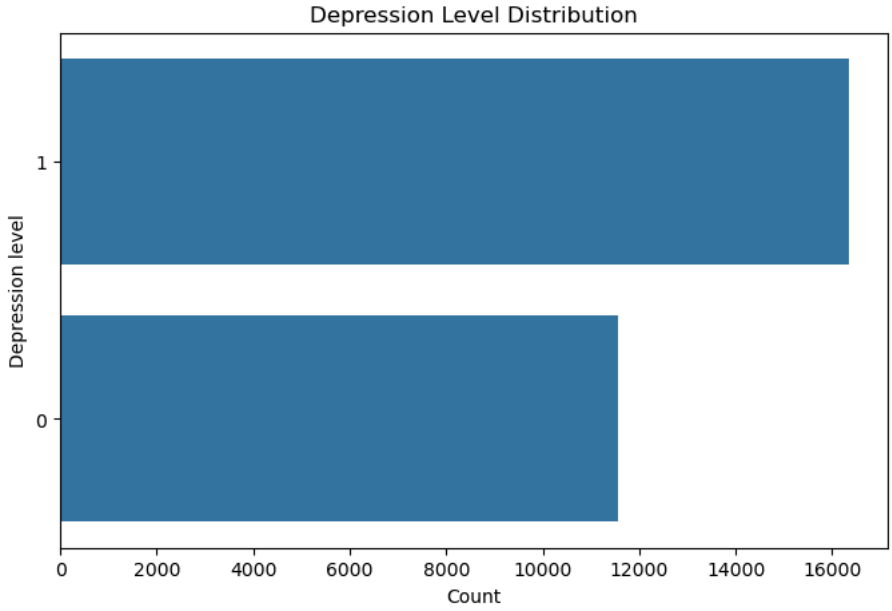
```
Gender Counts:
 Gender
Male      15547
Female    12354
```

```
Name: count, dtype: int64

Profession Counts:
 Profession
Student                  27870
Architect                    8
Teacher                      6
Digital Marketer             3
Content Writer               2
Chef                         2
Doctor                       2
Pharmacist                   2
Civil Engineer               1
UX/UI Designer               1
Educational Consultant       1
Manager                      1
Lawyer                       1
Entrepreneur                 1
Name: count, dtype: int64
```

```
import seaborn as sns

plt.figure(figsize=(8,5))
sns.countplot(data=df, y='Depression', order=df['Depression'].value_counts().index)
plt.title('Depression Level Distribution')
plt.xlabel('Count')
plt.ylabel('Depression level')
plt.show()
```




```
print(" Depression level", df['Depression'].value_counts().head(3))
```

```
 Depression level Depression
1    16336
0    11565
Name: count, dtype: int64
```

```
# Top locations
print("Academic Pressure", df['Academic Pressure'].value_counts().head(3))
```

```
Academic Pressure Academic Pressure
3.0    7462
5.0    6296
4.0    5155
Name: count, dtype: int64
```

```
print("Work Pressure", df['Work Pressure'].value_counts().head(3))
```

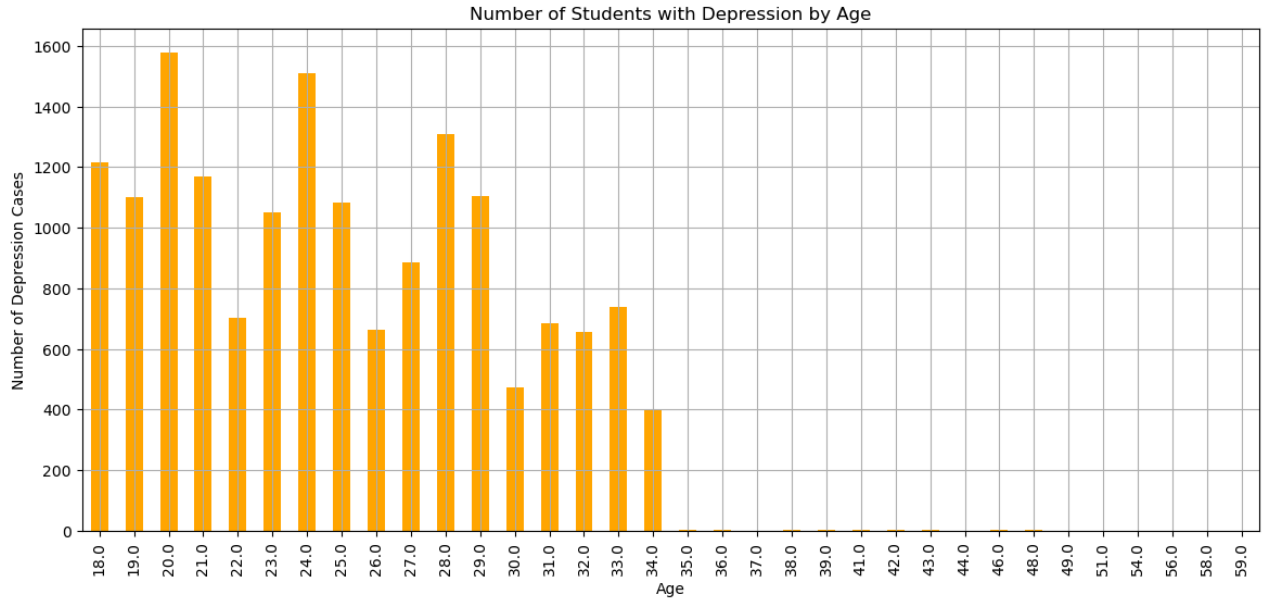
```
Work Pressure Work Pressure
0.0    27898
5.0        2
2.0        1
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt

# Group by age and count depression cases
```

```
depression_by_age = df.groupby('Age')['Depression'].sum()

plt.figure(figsize=(14, 6))
depression_by_age.plot(kind='bar', color='orange')
plt.title("Number of Students with Depression by Age")
plt.xlabel("Age")
plt.ylabel("Number of Depression Cases")
plt.grid(True)
plt.show()
```



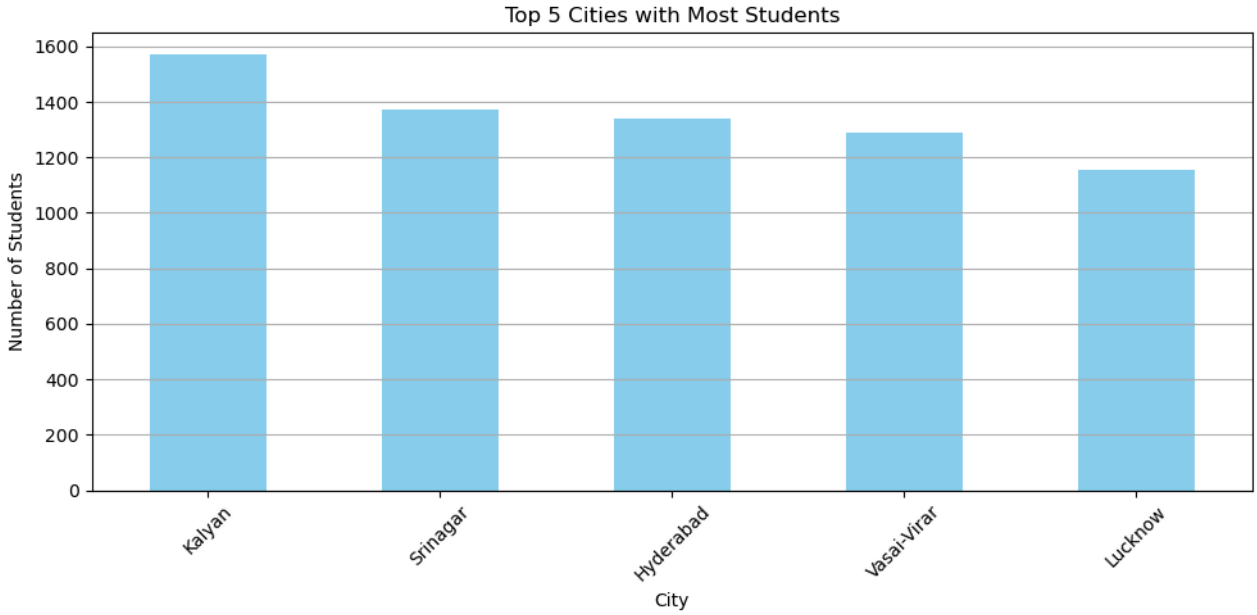

```
import matplotlib.pyplot as plt

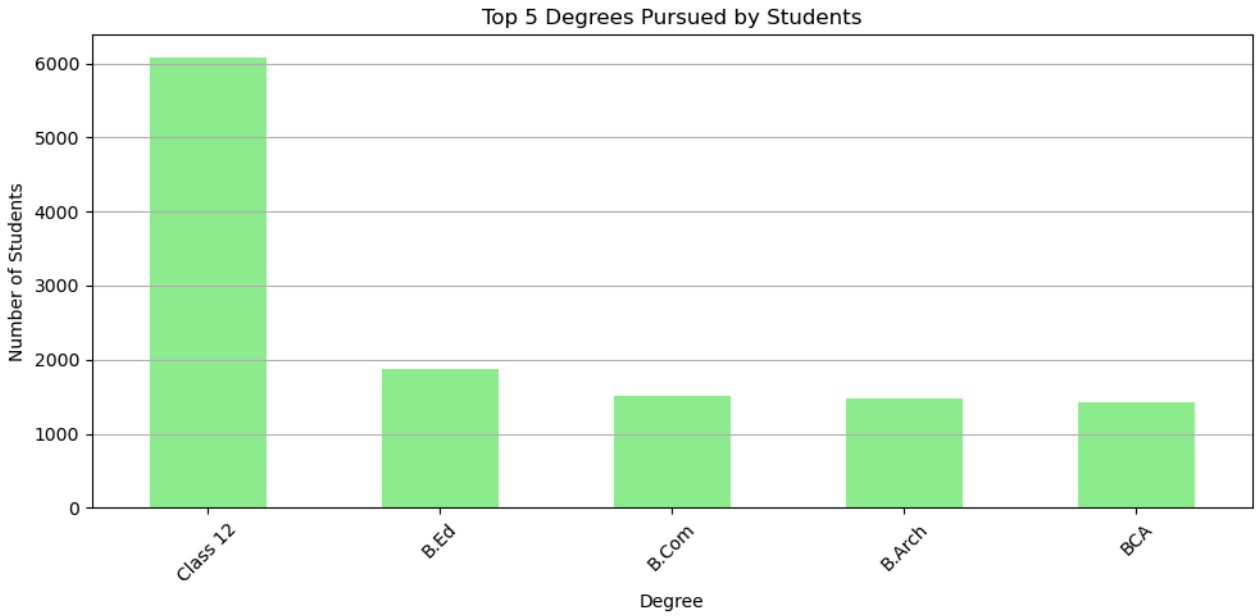
# Top 5 Cities with Most Students
top_cities = df['City'].value_counts().head(5)

plt.figure(figsize=(10, 5))
top_cities.plot(kind='bar', color='skyblue')
plt.title("Top 5 Cities with Most Students")
plt.xlabel("City")
plt.ylabel("Number of Students")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Top 5 Degrees Pursued by Students
top_degrees = df['Degree'].value_counts().head(5)

plt.figure(figsize=(10, 5))
top_degrees.plot(kind='bar', color='lightgreen')
plt.title("Top 5 Degrees Pursued by Students")
plt.xlabel("Degree")
plt.ylabel("Number of Students")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```






```
top_Degrees = df['Degree'].value_counts().head(5)
print("Top 5 Degrees:\n", top_Degrees)
```

```
Top 5 Degrees:
 Degree
Class 12    6080
B.Ed        1867
B.Com       1506
B.Arch      1478
BCA         1433
Name: count, dtype: int64
```

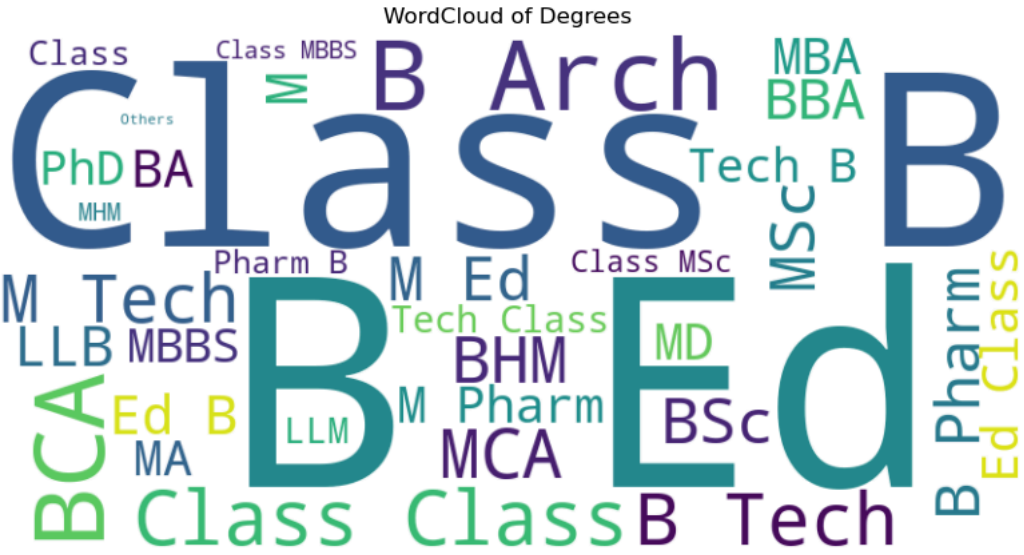
```
!pip install wordcloud
```

```
Requirement already satisfied: wordcloud in c:\users\user\anaconda3\lib\site-packages (1.9.4)
Requirement already satisfied: numpy>=1.6.1 in c:\users\user\anaconda3\lib\site-packages (from wordcloud) (1.26.4)
Requirement already satisfied: pillow in c:\users\user\anaconda3\lib\site-packages (from wordcloud) (10.4.0)
Requirement already satisfied: matplotlib in c:\users\user\anaconda3\lib\site-packages (from wordcloud) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1
Requirement already satisfied: cycler>=0.10 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.
Requirement already satisfied: fonttools>=4.22.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (
Requirement already satisfied: packaging>=20.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (24
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3
Requirement already satisfied: python-dateutil>=2.7 in c:\users\user\anaconda3\lib\site-packages (from matplotlib->wordcloud
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib-
```

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

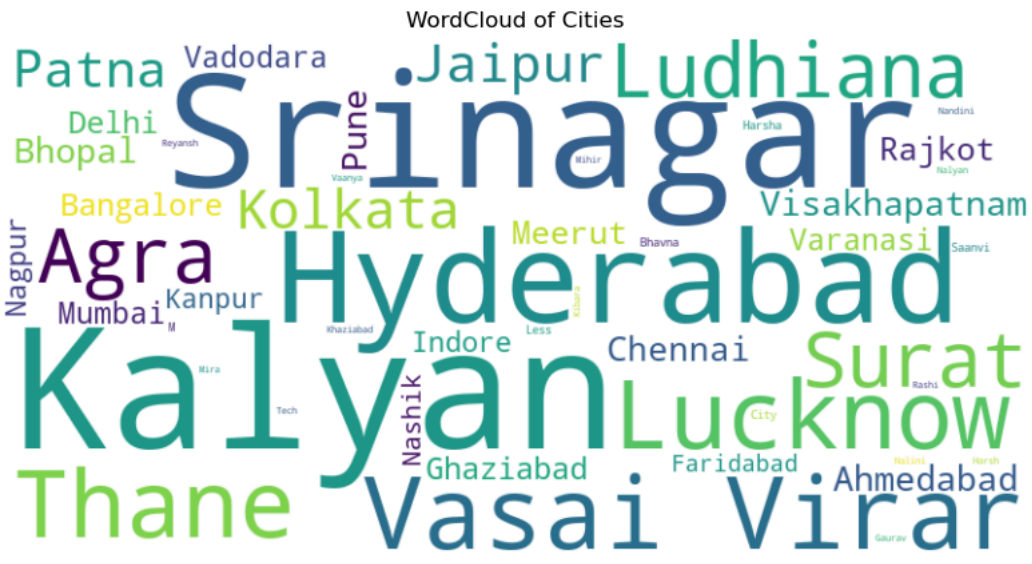
# Generate WordCloud from the Degree column
text = ' '.join(df['Degree'].dropna().astype(str).tolist())
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("WordCloud of Degrees")
plt.show()
```




```
text = ' '.join(df['City'].dropna().astype(str).tolist())
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("WordCloud of Cities")
plt.show()
```




```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Step 1: Create a synthetic 'data_collected_date' column with random dates
date_range = pd.date_range(start='2022-01-01', end='2024-12-31', freq='D')
df['data_collected_date'] = np.random.choice(date_range, size=len(df))
```

```
# Step 2: Convert to datetime (just to mimic real-world use)
df['data_collected_date'] = pd.to_datetime(df['data_collected_date'], errors='coerce')

# Step 3: Extract date-related features
df['collected_year'] = df['data_collected_date'].dt.year
df['collected_month'] = df['data_collected_date'].dt.month
df['collected_day'] = df['data_collected_date'].dt.day
df['collected_weekday'] = df['data_collected_date'].dt.weekday  # 0 = Monday, 6 = Sunday

# (Optional) Display the updated columns
df[['data_collected_date', 'collected_year', 'collected_month', 'collected_day', 'collected_weekday']].head()
```

	data_collected_date	collected_year	collected_month	collected_day	collected_weekday
0	2024-03-15	2024	3	15	4
1	2022-09-24	2022	9	24	5
2	2022-07-23	2022	7	23	5
3	2023-02-04	2023	2	4	5
4	2024-06-15	2024	6	15	5

```
df.head()
```

	id	Gender	Age	City	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	...	Have you ever had suicidal thoughts ?	Work/Stud Hour
0	2	Male	33.0	Visakhapatnam	Student	5.0	0.0	8.97	2.0	0.0	...	Yes	3.
1	8	Female	24.0	Bangalore	Student	2.0	0.0	5.90	5.0	0.0	...	No	3.
2	26	Male	31.0	Srinagar	Student	3.0	0.0	7.03	5.0	0.0	...	No	9.
3	30	Female	28.0	Varanasi	Student	3.0	0.0	5.59	2.0	0.0	...	Yes	4.
4	32	Female	25.0	Jaipur	Student	4.0	0.0	8.13	3.0	0.0	...	Yes	1.

```
import pandas as pd

# Load your dataset
df = pd.read_csv('Student Depression Dataset.csv')

# Step 1: Simulate a 'State' column (for demonstration purposes)
# You can skip this step if you only want to use 'City'
df['State'] = ['State_' + str(i % 5) for i in range(len(df))]  # Randomly assign 5 fake states

# Step 2: Combine 'City' and 'State' into a 'location' column
df['location'] = df['City'] + ', ' + df['State']

# Step 3: Frequency encoding of 'location'
location_freq = df['location'].value_counts().to_dict()
df['location_freq'] = df['location'].map(location_freq)

# (Optional) Show result
df[['City', 'State', 'location', 'location_freq']].head()
```

	City	State	location	location_freq
0	Visakhapatnam	State_0	Visakhapatnam, State_0	219
1	Bangalore	State_1	Bangalore, State_1	133
2	Srinagar	State_2	Srinagar, State_2	267
3	Varanasi	State_3	Varanasi, State_3	124
4	Jaipur	State_4	Jaipur, State_4	203

```
# Drop unnecessary columns after feature engineering
df.drop(columns=['City', 'State'], inplace=True)
```

```
df.head()
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have you ever had suicidal thoughts ?
0	2	Male	33.0	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Yes
1	8	Female	24.0	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	No
2	26	Male	31.0	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	Healthy	BA	No

```
# Check if Degree mentions PhD
df['mentions_phd'] = df['Degree'].str.contains('phd', case=False, na=False).astype(int)

# Check if Degree mentions M.Tech
df['mentions_mtech'] = df['Degree'].str.contains('m.tech', case=False, na=False).astype(int)

# Check if Degree mentions B.Tech
df['mentions_btech'] = df['Degree'].str.contains('b.tech', case=False, na=False).astype(int)
```

```
df.head()
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	...	Have you ever had suicidal thoughts ?	Work/Study Hours	Fi
0	2	Male	33.0	Student	5.0	0.0	8.97	2.0	0.0	5-6 hours	...	Yes	3.0	
1	8	Female	24.0	Student	2.0	0.0	5.90	5.0	0.0	5-6 hours	...	No	3.0	
2	26	Male	31.0	Student	3.0	0.0	7.03	5.0	0.0	Less than 5 hours	...	No	9.0	

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Apply Label Encoding to selected categorical columns
df['Gender'] = le.fit_transform(df['Gender'])
df['Profession'] = le.fit_transform(df['Profession'])
df['Dietary Habits'] = le.fit_transform(df['Dietary Habits'])
df['Degree'] = le.fit_transform(df['Degree'])
df['Sleep Duration'] = le.fit_transform(df['Sleep Duration'])
df['Have you ever had suicidal thoughts ?'] = le.fit_transform(df['Have you ever had suicidal thoughts ?'])
df['Family History of Mental Illness'] = le.fit_transform(df['Family History of Mental Illness'])

# If you created a 'location' column, encode that too
if 'location' in df.columns:
    df['location'] = le.fit_transform(df['location'])
```

```
df.head()
```

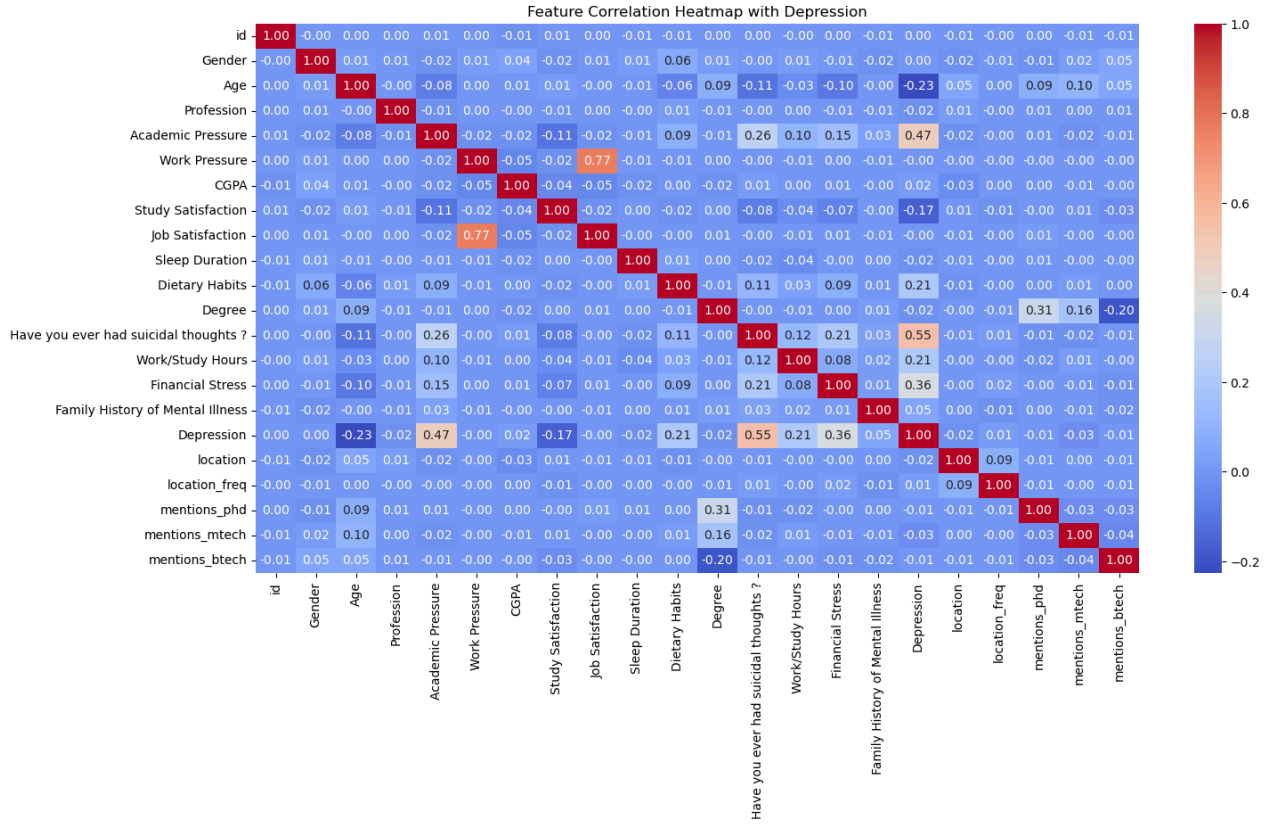
	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	...	Have you ever had suicidal thoughts ?	Work/Study Hours	Fi
0	2	1	33.0	11	5.0	0.0	8.97	2.0	0.0	0	...	1	3.0	
1	8	0	24.0	11	2.0	0.0	5.90	5.0	0.0	0	...	0	3.0	
2	26	1	31.0	11	3.0	0.0	7.03	5.0	0.0	2	...	0	9.0	
3	30	0	28.0	11	3.0	0.0	5.59	2.0	0.0	1	...	1	4.0	

```
import seaborn as sns
import matplotlib.pyplot as plt

# Compute correlation matrix (only numeric features)
```

```
correlation_matrix = df.corr(numeric_only=True)

# Plot heatmap
plt.figure(figsize=(16, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Feature Correlation Heatmap with Depression")
plt.show()
```




```
df.head()
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	...	Have you ever had suicidal thoughts ?	Work/Study Hours	Fi
0	2	1	33.0	11	5.0	0.0	8.97	2.0	0.0	0	...	1	3.0	
1	8	0	24.0	11	2.0	0.0	5.90	5.0	0.0	0	...	0	3.0	
2	26	1	31.0	11	3.0	0.0	7.03	5.0	0.0	2	...	0	9.0	
3	30	0	28.0	11	3.0	0.0	5.59	2.0	0.0	1	...	1	4.0	

```
import pandas as pd

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

# Strip whitespace from column names
```

```
df.columns = df.columns.str.strip()

# Print the actual column names to verify
print("Column names:", df.columns.tolist())

# Drop the specified columns safely
df.drop(columns=['City', 'Degree', 'Job Satisfaction'], inplace=True)

# Show first few rows to confirm
df.head()
```

```
Column names: ['id', 'Gender', 'Age', 'City', 'Profession', 'Academic Pressure', 'Work Pressure', 'CGPA', 'Study Satisfactio
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Sleep Duration	Dietary Habits	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress
0	2	Male	33.0	Student	5.0	0.0	8.97	2.0	5-6 hours	Healthy	Yes	3.0	1.0
1	8	Female	24.0	Student	2.0	0.0	5.90	5.0	5-6 hours	Moderate	No	3.0	2.0
2	26	Male	31.0	Student	3.0	0.0	7.03	5.0	Less than 5 hours	Healthy	No	9.0	1.0

```
df.head()
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Sleep Duration	Dietary Habits	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress
0	2	Male	33.0	Student	5.0	0.0	8.97	2.0	5-6 hours	Healthy	Yes	3.0	1.0
1	8	Female	24.0	Student	2.0	0.0	5.90	5.0	5-6 hours	Moderate	No	3.0	2.0
2	26	Male	31.0	Student	3.0	0.0	7.03	5.0	Less than	Healthy	No	9.0	1.0

```
# Step 1: Strip leading/trailing spaces from all column names
df.columns = df.columns.str.strip()

# Step 2: Apply frequency encoding to the 'City' column
df['location_encoded'] = df['location'].map(df['location'].value_counts())

# Step 3: (Optional) Show the first few rows to confirm the result
print(df[['location', 'location_encoded']].head())
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3805, in Index.get_loc(self, key)
   3804 try:
-> 3805     return self._engine.get_loc(casted_key)
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\\_libs\\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.PyObjectHashTable.get_item()

File pandas\\_libs\\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'location'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[94], line 5
      2 df.columns = df.columns.str.strip()
      4 # Step 2: Apply frequency encoding to the 'City' column
----> 5 df['location_encoded'] = df['location'].map(df['location'].value_counts())
      7 # Step 3: (Optional) Show the first few rows to confirm the result
      8 print(df[['location', 'location_encoded']].head())

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4102, in DataFrame.__getitem__(self, key)
   4100 if self.columns.nlevels > 1:
   4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
   4103 if is_integer(indexer):
   4104     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)
   3807     if isinstance(casted_key, slice) or (
   3808         isinstance(casted_key, abc.Iterable)
   3809         and any(isinstance(x, slice) for x in casted_key)
   3810     ):
   3811         raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
   3813 except TypeError:
   3814     # If we have a listlike key, _check_indexing_error will raise
   3815     #  InvalidIndexError. Otherwise we fall through and re-raise
   3816     #  the TypeError.
   3817     self._check_indexing_error(key)

KeyError: 'location'
```

```
df.head()
```

	id	Gender	Age	Profession	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Sleep Duration	Dietary Habits	Have you ever had suicidal thoughts ?	Work/Study Hours	Financial Stress
0	2	Male	33.0	Student	5.0	0.0	8.97	2.0	5-6 hours	Healthy	Yes	3.0	1.0
1	8	Female	24.0	Student	2.0	0.0	5.90	5.0	5-6 hours	Moderate	No	3.0	2.0
2	26	Male	31.0	Student	3.0	0.0	7.03	5.0	Less than	Healthy	No	9.0	1.0

```
import pandas as pd

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

# Strip whitespace from column names
df.columns = df.columns.str.strip()

# Define the column you want to drop
column_to_drop = 'Profession'

# Drop the column safely
if column_to_drop in df.columns:
    df.drop(columns=[column_to_drop], inplace=True)
    print(f"'{column_to_drop}' column dropped successfully.")
else:
    print(f"Column '{column_to_drop}' not found in the dataset.")

# Display the first few rows
df.head()
```

```
'Profession' column dropped successfully.
```

	id	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have yo ever ha suicida thought
0	2	Male	33.0	Visakhapatnam	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Ye
1	8	Female	24.0	Bangalore	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	N
2	26	Male	31.0	Srinagar	3.0	0.0	7.03	5.0	0.0	Less than	Healthy	BA	N

```
df.head()
```

	id	Gender	Age	City	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Sleep Duration	Dietary Habits	Degree	Have yo ever ha suicida thought
0	2	Male	33.0	Visakhapatnam	5.0	0.0	8.97	2.0	0.0	5-6 hours	Healthy	B.Pharm	Ye
1	8	Female	24.0	Bangalore	2.0	0.0	5.90	5.0	0.0	5-6 hours	Moderate	BSc	N
2	26	Male	31.0	Srinagar	3.0	0.0	7.03	5.0	0.0	Less than	Healthy	BA	N

```
# Define the target column
target = 'Depression'

# Create feature matrix X and target vector y
X = df.drop(columns=[target])
y = df[target]

# Display shapes of X and y
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (27901, 16)
y shape: (27901,)
```

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# Load the dataset
df = pd.read_csv("Student Depression Dataset.csv")

# Drop ID column
df = df.drop(columns=["id"])

# Drop rows with missing values
df = df.dropna()

# Encode categorical variables
label_encoders = {}
```